

Systems & Software Concepts

Theory notes and worked exercises, one concept at a time.

Leandro Dal Bo

CONTENTS

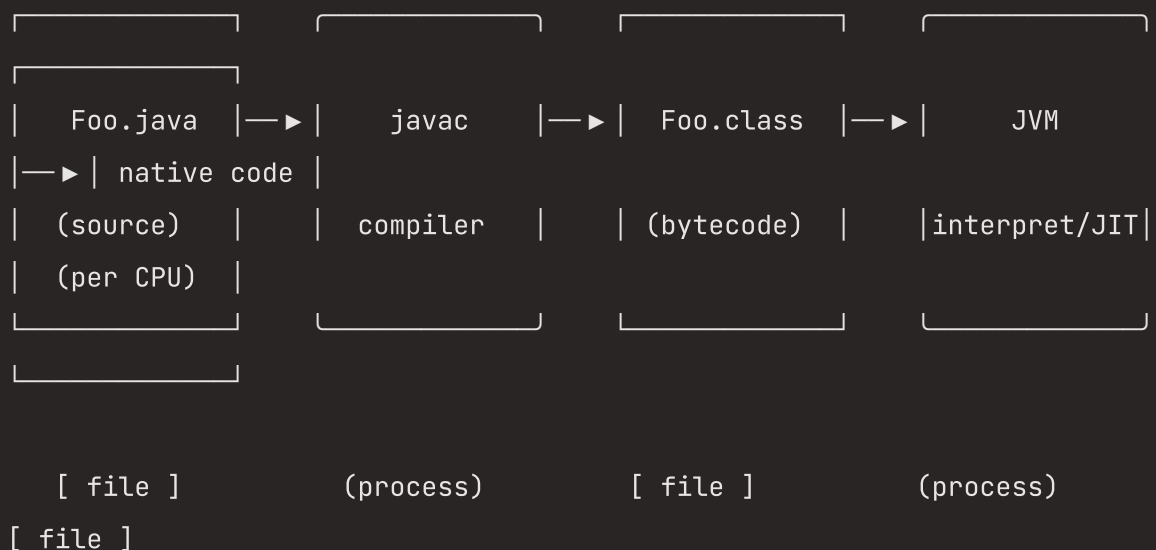
1. Entry 0: Inside the JVM
2. Entry 1: All Basic Concepts Together
3. Entry 2: Object Oriented Programming
4. Entry 2 extra: Collections, Enums, Exceptions & GC
5. Entry 3: SOLID and shiny day, also some design patterns
6. Entry 4: O Grande
7. Entry 5: Data Structures are important
8. Entry 6: Graphs and Trees
9. Entry 6 extra: Crazy extra bits and StringBuilder
10. Entry 7: System Design concepts
11. Entry 8: Load Balancers and Caching
12. Entry 9: Scaling the Database
13. Entry 10: System Design Concepts Mix
14. Entry 11: Spring Roots
15. Entry 12: Deep Spring
16. Entry 13: Real Software Products
17. Entry 14: Up & Running in Production
18. Entry 15: Relational Databases
19. Entry 16: Networking Basics
20. Entry 17: Operating Systems and Linux
21. Entry 18: TDCG, Using AI safely
22. Entry 19: Software Paradigms
23. Entry 20: Software Design, Writing It Well & Working Together

Entry 0: Inside the JVM

Platform independence: what do we actually mean?

"Write once, run anywhere" it means Java compiles to an intermediate format instead of compiling straight to a specific CPU's machine code.

- `javac` compiles your `.java` source into **bytecode** — a `.class` file. Bytecode is not native machine code; it's instructions for a virtual machine, the JVM.
- Any machine with a JVM built for its OS/CPU can load and run that same `.class` file. The portability lives in the JVM, not in your code. You ship one `.class` file, and a Mac, a Linux server, and a Windows box each run it through their own native JVM.



The JVM

The JVM (Java Virtual Machine) is the thing that actually executes your program. It's more than "just" a converter, its responsibilities include:

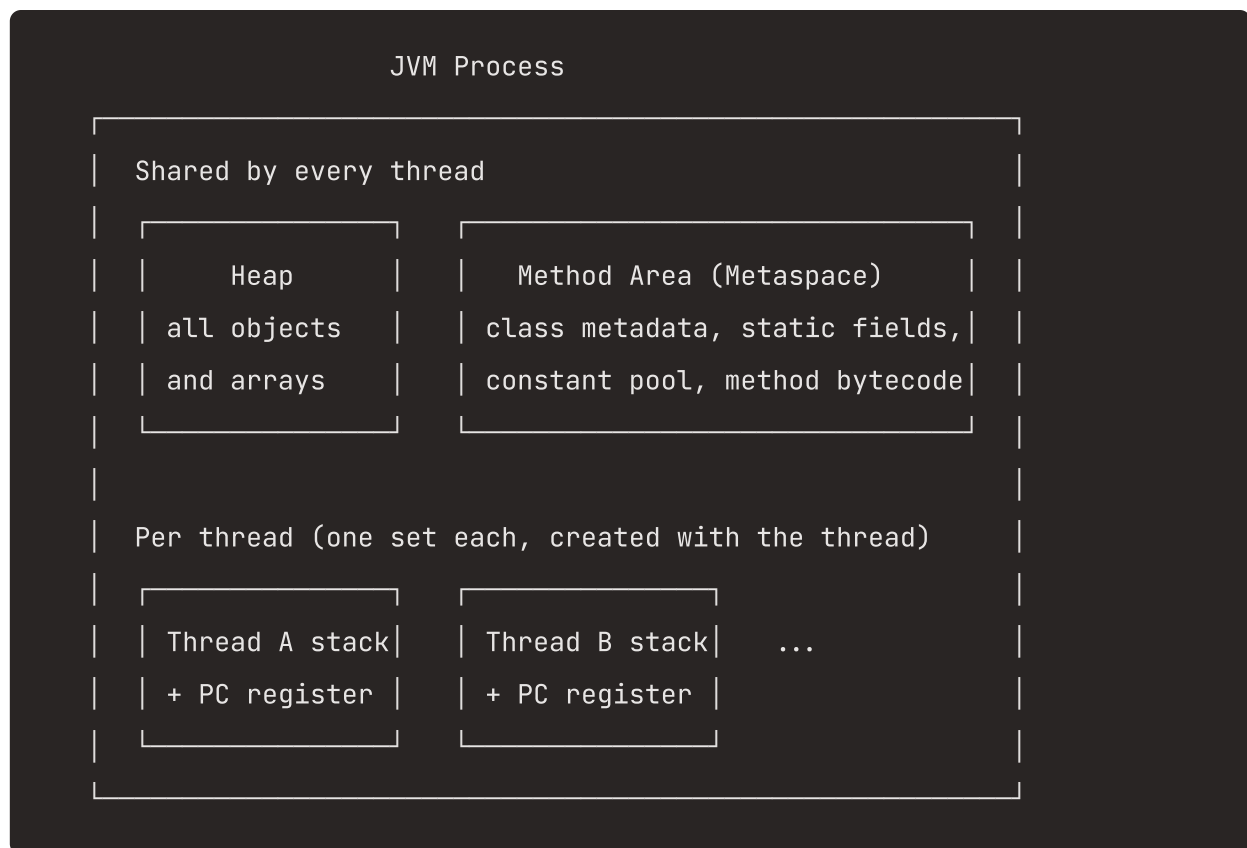
- **Class loading:** finds and loads `.class` files, verifies the bytecode is well-formed and safe before running it.
- **Execution:** runs the bytecode, initially by *interpreting* it instruction-by-instruction.
- **JIT compilation:** see below.
- **Memory management:** allocates and garbage-collects objects

JIT (Just-In-Time compiler)

The JVM compiles the bytecode to native machine code at runtime, then reuses that native version on later calls.

Memory: the JVM runtime data areas

Two of these areas are **shared by all threads**; two exist **per thread**. That distinction matters. This is the reason why the heap needs synchronization for thread safety and the stack doesn't.



- **Heap:** where every object and array actually lives, no matter which thread created it. Shared across all threads, garbage-collected. This is the memory people mean when they say "OutOfMemoryError: heap space."
- **Stack:** one per thread, not shared. Each method call pushes a new *stack frame* holding that call's local variables, method arguments, and intermediate results and the info needed to return to the caller. When the method returns, its frame is popped.
- **PC (Program Counter) register:** one per thread. Holds the address of the JVM instruction that thread is currently executing. Because each thread has its own PC, threads can each be in the middle of a different method at the same time without stepping on each other.

- **Method Area (Metaspace since Java 8):** shared, like the heap, but holds *class-level* data rather than object instances. The bytecode for each method, the runtime constant pool, ``static`` fields. Loaded once per class, not once per object.
- **Native method stack:** like the stack, but for native (non-Java, e.g. JNI/C) method calls. Mentioned for completeness. You won't touch this directly doing application-level Java.

Exercise: Palindrome Check

The solution fits in a few lines but still has a real complexity.

Problem

Write ``isAPalindrome(String word)``: return ``true`` if the string reads the same forwards and backwards, ``false`` otherwise.

```
"racecar" → true  
"hannah" → true  
"hello" → false
```

Simple enough to explain in one sentence to a beginner. The interesting part is *how* you check it, not *what* you're checking.

Tests (RED)

```
@Test
void shouldCheckPalindromes() {
    Day1 day1 = new Day1();

    assertThat(day1.isAPalindrome("racecar")).isTrue();
    assertThat(day1.isAPalindrome("hannah")).isTrue();
    assertThat(day1.isAPalindrome("hello")).isFalse();
}
```

Three cases: an odd-length palindrome, an even-length palindrome, and a plain rejection. That's enough to pin down the behavior before writing any implementation.

Implementation (GREEN)

```
/**
 * two pointers implementation
 * Space O(1)  $\neq$  String.reverse() O(N)
 * Time O(N)
 */
public boolean isAPalindrome(String word) {
    int x = 0;
    int y = word.length() - 1;

    while (x < y) {
        if (word.charAt(x)  $\neq$  word.charAt(y)) return false;
        x++;
        y--;
    }

    return true;
}
```

Why two pointers, not `reverse()`

The tempting first answer is: reverse the string, compare it to the original.

```
return word.equals(new StringBuilder(word).reverse().toString());
```

That works, but it allocates a whole new string just to throw it away. $O(n)$ extra space for a question that doesn't need it. Walking inward from both ends at once (one pointer at index `0`, one at `length - 1`) answers the same question in $O(n)$ time and $O(1)$ space: no extra allocation, no copy. It also stops early.

Whenever two characters disagree, you're done, you don't have to scan the whole string.

racecar

^		^	x=0, y=6: 'r' = 'r' → keep going	
	^		^	x=1, y=5: 'a' = 'a' → keep going
		^	^	x=2, y=4: 'c' = 'c' → keep going
			^	x=3, y=3: $x \geq y$, loop ends → true

Entry 1: All Basic Concepts Together

main method: The program starting point

```
public static void main(String[] args){}

|      |      |      |
|      |      |      | List of arguments passed to the program
|      |      |      |
|      |      | The name the JVM looks for, it must match this exactly
|      |      |
|      | Returns nothing to the JVM
|      |
| The JVM calls it without having to create an instance.
|
Must be reachable from outside the class, the JVM is the one calling
it
```

This exact signature is a contract with the JVM: change any part of it (rename `main`, drop `static`, make it return something) and the JVM won't recognize it as an entry point. It'll compile fine but fail at launch.

Pool of Strings

- A place in memory to store all the strings

- Strings are immutable, whenever a new string is created, the JVM will check the pool and if a matching string is found it will return the reference to that object.
- String literals like "foo" use the String pool. `new String("foo")` creates a new heap object, even though "foo" itself is pooled. So `new String("a") == "a"` is false, while `"a" == "a"` is true. Use `.intern()` to get the pooled reference.

Packages

- used to organize the code and prevent conflicts

Access Modifiers:

- `public`: access globally
- `private`: access only within the same class
- `protected`: access within the same package and subclasses
- default (no keyword): access only within the same package — narrower than `protected`, no subclass-outside-the-package access

Data Types

Primitives	size	wrapper
boolean	1 bit*	<code>Boolean</code>
char	2 bytes	<code>Character</code>
byte	1 byte	<code>Byte</code>
short	2 bytes	<code>Short</code>
int	4 bytes	<code>Integer</code>
long	8 bytes	<code>Long</code>
float	4 bytes	<code>Float</code>
double	8 bytes	<code>Double</code>

Wrappers encapsulate a primitive in an object, so they're usable where an object is required (generics, collections — you can't have a `List<int>`, only `List<Integer>`). They're `final` and immutable, same as `String`.

Non-primitives (reference types): `String`, arrays, and any `class` you define. A variable of a reference type holds a reference to an object on the heap, not the value itself — that's the real primitive/non-primitive split, not the size.

String

- An abstraction to store a sequence of characters

- Immutable — every "modification" (``concat``, ``substring``, ...) returns a new ``String`` rather than changing the original. That's what makes string pooling and safe sharing across threads possible in the first place.
- `StringBuffer` thread safe (synchronized methods)
- `StringBuilder` not thread safe, but faster — use it unless multiple threads touch the same instance

Array:

- A data structure to store a sequence of elements
- It can have multiple dimensions (matrix)
- Elements can be accessed using the index ==> $O(1)$
- size is fixed

Class Variable vs Instance Variable

- Class variables are ``static``. One copy shared by every instance of that class.
- Instance variables are non-``static``. Each instance gets its own copy; it's part of that object's state. (Access modifier is a separate, independent choice — ``private`` is just the usual convention for encapsulation, not what makes it an instance variable.)

Constructors

- Special methods used to create an instance of a class
- Same name as the class, no return type (not even `void`)
- If you don't write one, the compiler generates a no-arg default constructor
- Can be overloaded (multiple constructors, different parameter lists) to support different ways of building the object

Volatile Keyword

- Every read/write goes straight to main memory instead of a thread-local CPU cache, so a write on one thread is immediately visible to reads on other threads.
- It only guarantees **visibility**, not **atomicity**. `count++` on a `volatile int` is still read-modify-write — two threads can interleave and lose an update. For that you need `synchronized` or `AtomicInteger`, not `volatile`.

I/O Packages

- Perform Input output operations (read/write)
- Buffered streams reduce the number of operation to interact with devices

Exercise: Prime Check

A simple problem and a good excuse to talk about why `isPrime` should never loop all the way to `n`.

Problem

Write `isPrime(int number)`: return `true` if the number is prime or `false` otherwise.

Prime Number: greater than 1, divisible only by 1 and itself

```
2 → true
13 → true
29 → true
4 → false
12 → false
25 → false
```

Tests (RED)

```
@Test
public void shouldCheckIsPrime() {
    Day1 d1 = new Day1();
    assertThat(d1.isPrime(2)).isTrue();
    assertThat(d1.isPrime(13)).isTrue();
    assertThat(d1.isPrime(29)).isTrue();
    assertThat(d1.isPrime(4)).isFalse();
    assertThat(d1.isPrime(12)).isFalse();
    assertThat(d1.isPrime(25)).isFalse();
}
```

Implementation (GREEN)

```
public boolean isPrime(int number) {
    if (number ≤ 1) {
        return false;
    }

    for (int i = 2; i ≤ Math.sqrt(number); i++) {
        if (number % i == 0) return false;
    }

    return true;
}
```

Why stop at $\sqrt{n}$

Factors come in pairs. If $n = a \times b$, at least one factor must be $\leq \sqrt{n}$.

So once we've checked every number up to $\sqrt{n}$ and found no divisor, there can't be a larger divisor either — it would have a smaller matching factor that we'd already have checked.

```
25 = 1 x 25
    = 5 x 5      ← sqrt(25) = 5, the pair meets exactly here
```

For 25, we only need to check 2 through 5. This reduces the loop from $O(n)$ to $O(\sqrt{n})$.

Entry 2: Object Oriented Programming

Objects

- Every object is an instance of a class.
- An object has **state** and **behaviour**.
- State is made up of the instance fields.
- Behaviour is defined through methods.

Classes

- A class is the template, or blueprint, used to create objects.
- ``this`` refers to the current object.
- ``super`` refers to the parent class (its constructor, fields, or methods).

Abstraction

- Achieved using classes and interfaces.
- A class should expose only what's relevant to its users — the focus is on the "WHAT", not the "HOW".

- The user of a class needs to know its interface (the public contract), not its implementation details.
- Promotes loose coupling and high cohesion.

Encapsulation

- Implemented by keeping an object's fields ``private`` and exposing access through methods (getters/setters, or better, behaviour that operates on the state directly).
- Hides the object's internal state, so the internal representation can change without breaking callers.

Inheritance

- Lets a class be based on an existing one (``extends``) — the subclass gets the parent's fields and methods.
- Creates tight coupling: a change in the parent can ripple into every subclass, and deep hierarchies get hard to reason about and maintain. This is why "favor composition over inheritance" is a common piece of design advice — model "has-a" with a field instead of "is-a" with ``extends`` unless the is-a relationship is genuine and stable.

Polymorphism

Gives objects the ability to be treated through a common type while behaving differently underneath. Two flavors:

- **Overloading** — same method name, different parameter list. Resolved at **compile time** based on the declared argument types.
- **Overriding** — a subclass redefines a method it inherited. Resolved at **runtime**, based on the actual object type (dynamic dispatch).

UML class diagram: all four pillars in one example

- `Shape` is abstract: it declares the contract (`area()`) without providing an implementation — abstraction.
- `color` is `protected`, accessible to subclasses but hidden from the outside world — encapsulation.
- `Circle` and `Rectangle` both `extend Shape` — inheritance.
- Calling `shape.area()` on a `Shape` reference runs `Circle`'s or `Rectangle`'s version depending on the actual object at runtime — polymorphism (dynamic dispatch).

Covariant return types

A subclass overriding a method may narrow the return type to a subtype of what the parent declared:

```
class Animal {
    Animal reproduce() { return new Animal(); }
}

class Dog extends Animal {
    @Override
    Dog reproduce() { return new Dog(); } // narrows Animal → Dog
}
```

This is still a valid override (not overloading) because the return type is a subtype — callers that expect an `Animal` back are never surprised, they just might get a more specific one.

Abstract class vs interface

Aspect	Abstract class	Interface
Instantiation	Cannot be instantiated directly	Cannot be instantiated directly
Constructors	Can have constructors, run via <code>super()</code> when a subclass is built	No constructors
Fields	Can hold real instance state, any access modifier	Fields are implicitly <code>public static final</code> — constants only, no instance state
Methods	Can mix abstract and concrete methods; zero abstract methods is legal	Before Java 8: every method implicitly abstract. Java 8+: <code>default</code> / <code>static</code> methods allowed
Inheritance	Single inheritance only (<code>extends</code> one class)	A class can implement multiple interfaces
When to use	Related types that share state and some implementation	An unrelated capability/contract that different types can opt into

A common misconception: an abstract class does **not** need to declare any abstract method — `abstract` just means "cannot be instantiated," even if every method already has a body.

Exercise: GCD & LCM

The Euclidean algorithm — one of the oldest algorithms still in everyday use (Euclid's *Elements*, ~300 BC) — and the LCM identity that rides on top of it.

Problem

Write `mcd(int a, int b)`: return the greatest common divisor of `a` and `b` (the largest number that divides both evenly).

```
mcd(12, 8)  → 4  
mcd(12, 18) → 6  
mcd(36, 48) → 12
```

Write `lcm(int a, int b)`: return the least common multiple of `a` and `b` (the smallest number both divide evenly into).

```
lcm(4, 6)   → 12  
lcm(5, 7)   → 35  
lcm(15, 20) → 60
```

Tests (RED)

```
@Test
public void shouldCheckGCD() {
    Day1 d1 = new Day1();
    assertEquals(d1.mcd(12, 8), 4);
    assertEquals(d1.mcd(12, 18), 6);
    assertEquals(d1.mcd(36, 48), 12);
}
```

```
@Test
public void shouldCheckLCM() {
    Day1 d1 = new Day1();
    assertEquals(d1.lcm(4, 6), 12);
    assertEquals(d1.lcm(5, 7), 35);
    assertEquals(d1.lcm(15, 20), 60);
}
```

Implementation (GREEN)

```
/**
 * Euclidean algorithm using subtraction
 */
public int mcd(int a, int b) {
    while (a != b) {
        if (a < b) {
            b = b - a;
        } else {
            a = a - b;
        }
    }
    return a;
}

public int lcm(int a, int b) {
    return (a * b) / mcd(a, b);
}
```

Why repeated subtraction works

The core fact: any number that divides both a and b also divides their difference. So $\text{gcd}(a, b) = \text{gcd}(a - b, b)$ when $a > b$ — shrinking the pair never changes the answer, it just gets there faster. Keep subtracting the smaller from the larger and the pair converges until $a = b$, and that shared value is the GCD.

```
mcd(12, 8)
  12, 8 → a > b: a = 12 - 8 = 4
  4, 8 → b > a: b = 8 - 4 = 4
  4, 4 → equal, done → 4
```

Subtraction vs modulo — the complexity angle worth mentioning out loud

The version above is the textbook *subtractive* Euclidean algorithm: easy to reason about, but its worst case is $O(\max(a, b))$ — think `mcd(1, 1_000_000)`, which subtracts 1 a million times.

The standard library version (and the one worth using in practice) swaps subtraction for modulo:

```
public int mcdFast(int a, int b) {
    return b == 0 ? a : mcdFast(b, a % b);
}
```

`a % b` does in one step what the subtractive version does in `a / b` steps, so this converges in $O(\log(\min(a, b)))$ — the same correctness argument, dramatically fewer iterations.

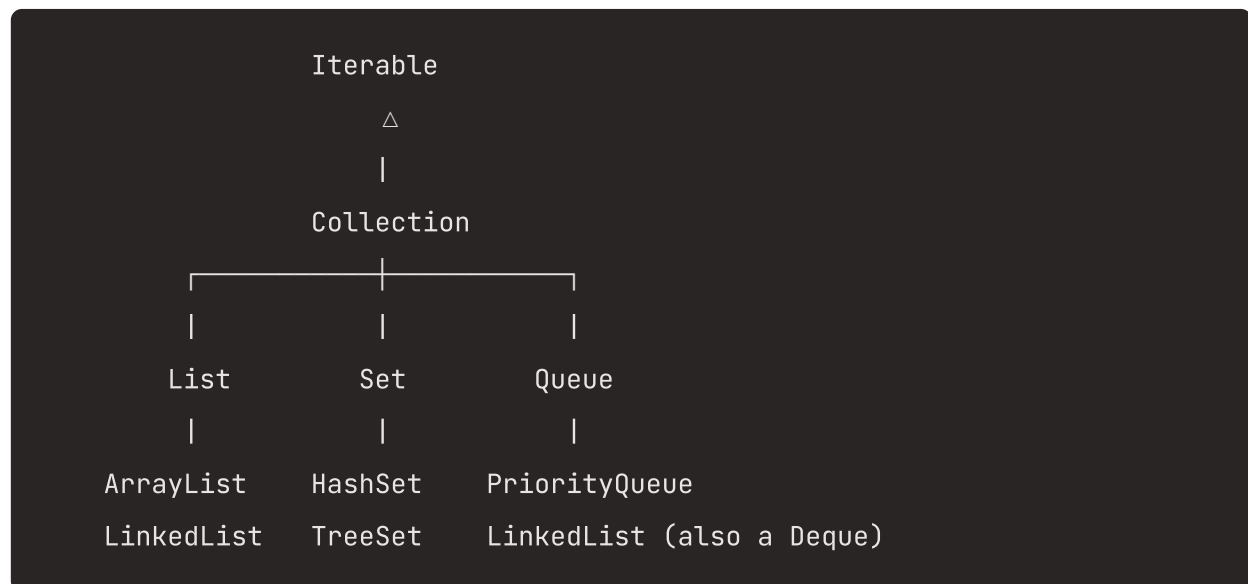
LCM: built on GCD, not independent of it

`lcm(a, b) = (a * b) / gcd(a, b)` falls out of a simple identity: `gcd(a, b) * lcm(a, b) = a * b`. It's cheap once `mcd` exists — no need to re-derive LCM from scratch.

Entry 2 extra: Collections, Enums, Exceptions & GC

Java Collections Framework

A unified set of interfaces and classes for storing and manipulating groups of objects.



`Map` is not a `Collection` — it stores key-value pairs, not a sequence of single elements, so it sits in its own branch of the hierarchy with `HashMap`, `LinkedHashMap`, `TreeMap`.

- **List** — allows duplicates, indexable. `ArrayList` is backed by a resizable array ($O(1)$ get by index, $O(n)$ insert/remove in the middle). `LinkedList` is a doubly linked list ($O(1)$ insert/remove at the ends, $O(n)$ get by index).

- **Set** — no duplicates. `HashSet` (no order guarantee, $O(1)$ average add/contains via hashing), `LinkedHashSet` (preserves insertion order), `TreeSet` (sorted, $O(\log n)$ operations, backed by a red-black tree).
- **Queue** — FIFO by default (`LinkedList` used as a `Queue`). `PriorityQueue` is the exception: elements come out in priority order, not insertion order, so it's a min-heap under the hood, not a FIFO structure.
- **Map** — key-value pairs, keys unique. Same `Hash`/`LinkedHash`/`Tree` split as `Set` (a `HashSet` is, in fact, implemented as a `HashMap` under the hood with dummy values).
- **Iterator** — the contract every `Collection` implements to be walked element by element without exposing its internal structure:

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String value = it.next();
    if (shouldRemove(value)) {
        it.remove(); // safe removal during iteration
    }
}
```

Enums

A type-safe way to represent a fixed set of constants — instead of `int` or `String` constants (which the compiler can't validate), an `enum` gives you a closed set of values the compiler *does* check.

```
public enum Day {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;  
  
    public boolean isWeekend() {  
        return this == SATURDAY || this == SUNDAY;  
    }  
}
```

Each constant is a singleton instance of the enum type, so `enum` values can carry fields, constructors, and methods — not just names. A `switch` over an enum lets the compiler warn when a case is missing, something plain `int`/`String` constants can never give you.

Exceptions

An event that interrupts a program's normal execution. It can be handled where it occurs, or propagated up the call stack for a caller to handle.

Category	When it's known	Examples	Must be declared/caught?
Checked	Compile time	<code>IOException</code> , <code>SQLException</code>	Yes — <code>throws</code> or <code>try/catch</code> , enforced by the compiler
Unchecked	Runtime	<code>NullPointerException</code> , <code>ArrayIndexOutOfBoundsException</code>	No — extends <code>RuntimeException</code> , compiler doesn't enforce handling
Error	Runtime, usually unrecoverable	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>	No — not meant to be caught, the program generally can't recover

```
try {
    riskyOperation();
} catch (IOException e) {
    // handle or wrap and rethrow
} finally {
    // always runs, even if a return happens above
}
```

`Checked` vs `Unchecked` is a design choice about the exception class itself (does it extend `Exception` or `RuntimeException`), not about the situation that caused it — that's why the compiler can enforce checked exceptions but not unchecked ones.

Garbage Collection

The JVM automatically reclaims heap memory occupied by objects that no longer have any reachable reference — no manual `free()` like in C.

- An object becomes eligible for collection once nothing reachable from a GC root (local variables on the stack, static fields, active threads) points to it, directly or transitively.
- Collection runs on a **daemon thread**: a background thread that doesn't keep the JVM alive by itself, so the program can still exit normally even while GC is running.
- Most JVMs use **generational** collection: new objects go into a small "young generation" that's collected frequently and cheaply (most objects die young); objects that survive several collections get promoted to the "old generation," collected less often. This split is why GC is usually fast in practice despite scanning a large heap — most of the heap is rarely touched.

Exercise: Factorial & Fibonacci

Two of the oldest recursion problems, both done iteratively. A good excuse to compare the iterative and recursive shapes of the same idea.

Problem

Write `factorial(int x)`: return `x!` (`x * (x-1) * (x-2) * ... * 1`).

```
factorial(5) → 120
```

Write `fibonacci(int x)`: return an array holding the first `x` numbers of the Fibonacci sequence (each number is the sum of the two before it, starting `0, 1`).

```
fibonacci(5) → [0, 1, 1, 2, 3]
```

Tests (RED)

```
@Test
public void shouldCalculateFactorial() {
    Day2 day2 = new Day2();
    assertThat(day2.factorial(5)).isEqualTo(120);
}

@Test
public void shouldCalculateFibonacciForN() {
    Day2 day2 = new Day2();
    assertThat(day2.fibonacci(5)).isEqualTo(new int[]{0, 1, 1, 2, 3});
}
```

Implementation (GREEN)

```
/*  
    factorial:  $x! = x \times (x-1) \times (x-2) \times \dots$   
    Time complexity:  $O(x)$   
    Space complexity:  $O(1)$   
*/  
public int factorial(int x) {  
    int result = 1;  
  
    for (int i = x; i  $\geq$  1; i--) {  
        result *= i;  
    }  
  
    return result;  
}
```

```

/*
    Complexity: O(x) time and O(x) space
*/
public int[] fibonacci(int x) {
    int[] result = new int[x];
    int a = 0;
    int b = 1;

    if (x == 0) return new int[]{0};
    if (x == 1) {
        return new int[]{0, 1};
    }

    result[a] = a;
    result[b] = b;

    for (int i = 2; i < x; i++) {
        result[i] = result[a] + result[b];
        a++;
        b++;
    }

    return result;
}

```

Factorial: iterative vs recursive

The textbook definition of factorial is recursive ($x! = x \times (x-1)!$), so the recursive version reads closest to the math:

```
public int factorialRecursive(int x) {  
    if (x ≤ 1) return 1;  
    return x * factorialRecursive(x - 1);  
}
```

Same $O(x)$ time either way, but the recursive version requires $O(x)$ space for the call stack — one frame per pending multiplication — while the iterative loop above stays $O(1)$ space.

Fibonacci: why keep the whole array

`fibonacci` here returns the *sequence*, not just the `x`-th value, so it genuinely needs $O(x)$ space — every previous value stays in the output. If the problem only asked for the `x`-th Fibonacci number, the usual next question is "can you do it without an array?" — yes, since each step only ever needs the previous two values:

```
public int fibonacciNth(int x) {  
    if (x == 0) return 0;  
  
    int prev = 0;  
    int curr = 1;  
  
    for (int i = 2; i ≤ x; i++) {  
        int next = prev + curr;  
        prev = curr;  
        curr = next;  
    }  
  
    return curr;  
}
```

That drops space from $O(x)$ to $O(1)$.

Entry 3: SOLID and shiny day, also some design patterns

SOLID

Managing **coupling and cohesion**: related things grouped together (cohesion), unrelated things kept from leaning on each other's internals (coupling).

Single Responsibility Principle (S)

A class should have only one reason to change. Uncle Bob's framing: a module should answer to **one** actor/part of the business, if two different stakeholders could ask for a change to the same class for unrelated reasons, it's doing too much.

```
// violates SRP: persistence + formatting + business rules, all in one
class
class Invoice {
    double calculateTotal() { /* ... */ }
    void saveToDatabase() { /* ... */ }
    String formatAsPdf() { /* ... */ }
}

// SRP: each reason to change gets its own class
class Invoice {
    double calculateTotal() { /* ... */ }
}
class InvoiceRepository {
    void save(Invoice invoice) { /* ... */ }
}
class InvoicePdfFormatter {
    String format(Invoice invoice) { /* ... */ }
}
```

Open/Closed Principle (O)

A class should be **open for extension, closed for modification**. Add new behaviour by writing new code (a new subclass/implementation), not by editing code that already works and is already tested.

```
// violates OCP: every new shape means editing this method again
double area(Object shape) {
    if (shape instanceof Circle c) return Math.PI * c.radius() *
c.radius();
    if (shape instanceof Rectangle r) return r.width() * r.height();
    // adding Triangle means coming back here
    return 0;
}

// OCP: each shape knows its own area, area() never changes again
interface Shape {
    double area();
}

class Circle implements Shape {
    public double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    public double area() { return width * height; }
}
```

Liskov Substitution Principle (L)

Wherever code expects the base type, a subtype must be usable in its place **without breaking correctness**. A subclass shouldn't strengthen preconditions or weaken postconditions the caller relies on.

Interface Segregation Principle (I)

Don't force a class to implement methods it has no use for. Many small, focused interfaces beat one large one.

Dependency Inversion Principle (D)

High-level modules (business logic) shouldn't depend on low-level modules (implementation details like a specific database or HTTP client), both should depend on an **abstraction**. This is what makes the low-level piece swappable and the high-level piece testable in isolation.

```
// violates DIP: the service is welded to one concrete implementation
class NotificationService {
    private final EmailSender sender = new EmailSender();
    void notify(String msg) { sender.send(msg); }
}

// DIP: depend on an interface, the concrete class is injected
interface MessageSender {
    void send(String msg);
}

class NotificationService {
    private final MessageSender sender;
    NotificationService(MessageSender sender) { this.sender = sender; }
    void notify(String msg) { sender.send(msg); }
}
```

`EmailSender` or `SmsSender` can now be swapped — or replaced with a mock in a test — without touching `NotificationService`.

Useful Design Patterns

State

The behaviour of an object changes depending on its internal state, without a wall of ``if`/`switch`` statements at every call. Each state is its own class implementing a shared interface, and the object delegates to whichever one it currently holds.

Strategy

Lets an algorithm be swapped at runtime by depending on an interface instead of a hard-coded implementation. Structurally identical to State, but the intent differs: Strategy picks *how* to do something once, State reacts to transitions *over time*.

```
interface DiscountStrategy {
    double apply(double price);
}

class NoDiscount implements DiscountStrategy {
    public double apply(double price) { return price; }
}

class TenPercentOff implements DiscountStrategy {
    public double apply(double price) { return price * 0.9; }
}

class Checkout {
    private final DiscountStrategy discount;

    Checkout(DiscountStrategy discount) { this.discount = discount; }

    double total(double price) { return discount.apply(price); }
}
```

Swapping `new NoDiscount()` for `new TenPercentOff()` at construction time changes the checkout's behaviour with zero changes to `Checkout` itself. Same shape as the Open/Closed example above.

Decorator

Adds behaviour to an individual object at runtime by wrapping it in another object that shares its interface. An alternative to subclassing when the combination of behaviours needed would otherwise require a new subclass for every combination.

```
interface Coffee {
    double cost();
}

class BlackCoffee implements Coffee {
    public double cost() { return 2.0; }
}

class WithMilk implements Coffee {
    private final Coffee inner;

    WithMilk(Coffee inner) { this.inner = inner; }

    public double cost() { return inner.cost() + 0.5; }
}

class WithSugar implements Coffee {
    private final Coffee inner;

    WithSugar(Coffee inner) { this.inner = inner; }

    public double cost() { return inner.cost() + 0.2; }
}

// stack decorators to combine behaviour: 2.0 + 0.5 + 0.2 = 2.7
Coffee order = new WithSugar(new WithMilk(new BlackCoffee()));
order.cost();
```

Facade

A single, simplified interface in front of a complex subsystem. The caller talks to the facade and never sees the tangle of classes it coordinates underneath.

Factory

Wraps the constructor call behind a method, so the caller asks for "a thing that satisfies this interface" instead of naming a concrete class directly. It is useful when construction involves choosing between several implementations or needs setup logic beyond `new`.

Builder

Constructs an object step by step through a chain of setters, useful when a constructor would otherwise need a long list of parameters. Each call returns `this`, so calls read as a fluent chain ending in `.build()`.

Exercise: Longest Substring Without Repeating Characters

A sliding-window problem.

Problem

Write `longestSubstring(String input)`: return the length of the longest substring that has no repeated characters.

```
"abcabcbb" → "abc" → 3  
"bbbbbb" → "b" → 1  
"pwwkew" → "wke" → 3
```

Tests (RED)

```
@Test  
public void shouldFindLongestSubstringWithoutRepeats() {  
    Day3 d3 = new Day3();  
  
    assertThat(d3.longestSubstring("abcabcbb")).isEqualTo(3);  
    assertThat(d3.longestSubstring("bbbbbb")).isEqualTo(1);  
    assertThat(d3.longestSubstring("pwwkew")).isEqualTo(3);  
    assertThat(d3.longestSubstring("")).isEqualTo(0);  
}
```

Implementation (GREEN)

```
/*
    sliding window / two-pointer technique
    The current window must contain only unique characters.

     $O(N + N) = O(2N) \Rightarrow O(N)$ 
*/
public int longestSubstring(String input) {
    Set<Character> characters = new HashSet<>();

    int left = 0;
    int max = 0;

    for (int right = 0; right < input.length(); right++) {
        while (characters.contains(input.charAt(right))) {
            characters.remove(input.charAt(left));
            left++;
        }

        characters.add(input.charAt(right));
        max = Math.max(max, right - left + 1);
    }

    return max;
}
```

Why a window, not brute force

The brute-force approach checks every substring for uniqueness — $O(n^3)$ ($O(n^2)$ substrings, $O(n)$ each to verify). The sliding window gets to $O(n)$ by never re-

examining a character it doesn't have to:

- **`right`** walks forward once, always expanding the window by one character per step.
- **`left`** only moves forward too — when the character at **`right`** is already in the window, **`left`** advances (shrinking from the left) until the duplicate is gone.
- Because both pointers only move forward and each visits every index at most once, the whole scan is $O(2n) \rightarrow O(n)$, not $O(n)$ work repeated per starting point.

The **`Set<Character>`** mirrors exactly what's inside the current window, so "does the window already contain this character" is an $O(1)$ check instead of re-scanning the window itself.

Entry 4: O Grande

What Big O actually measures

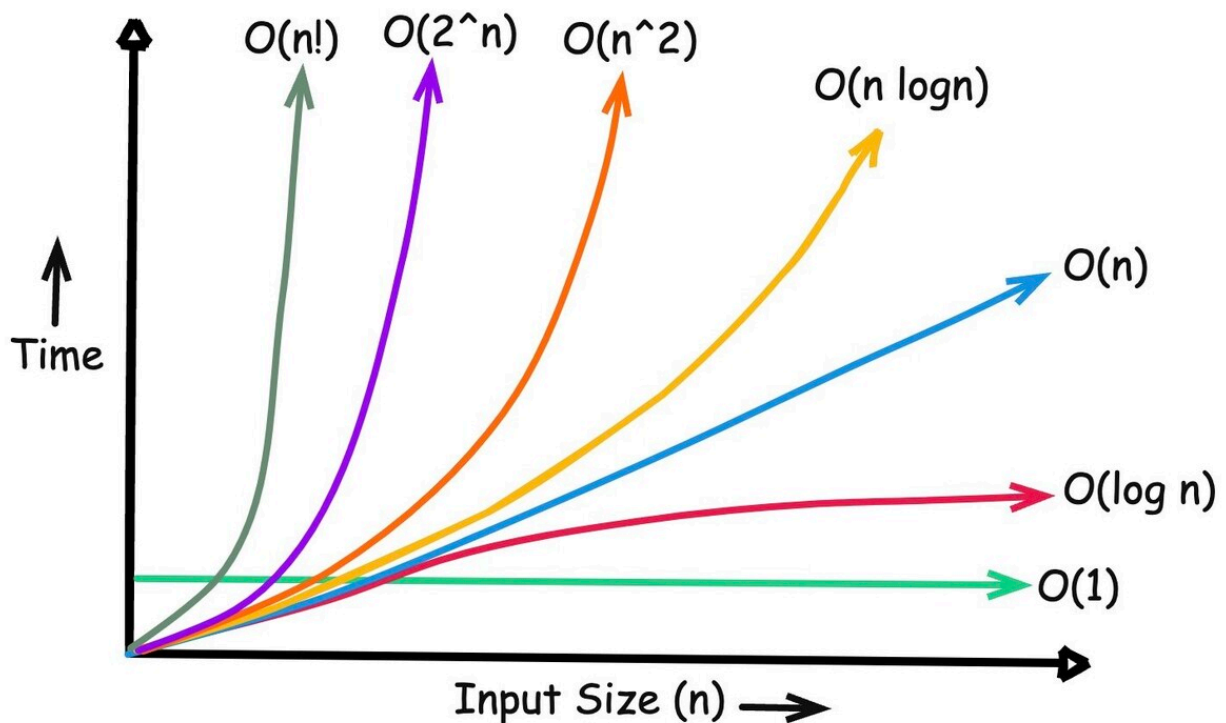
How the runtime (or memory) scales as the input gets bigger?

Big O describes the **cost of an algorithm** as the input grows. An $O(n)$ algorithm on a slow laptop can beat an $O(n^2)$ algorithm on a supercomputer once ``n`` is large enough, because the $O(n^2)$ one is doing fundamentally more work per extra element.

Time complexity vs space complexity

- **Time complexity:** the number of *operations* grows with ``n``.
- **Space complexity:** the memory needed by the algorithm beyond the input itself, as ``n`` grows.

The complexity classes



- **$O(1)$ — constant.** Same cost regardless of n . For example Array index access (`arr[i]`), a HashMap `get`.
- **$O(\log n)$ — logarithmic.** The problem shrinks by a constant *fraction* each step. Binary search: each comparison halves the remaining search space.

`N = 16`

`N → 8 → 4 → 2 → 1` (4 halvings to go from 16 to 1)

$16 = 2^4$, so it took 4 steps — in general, n elements take $\log_2 n$ steps. Doubling the input only adds **one more step**, which is what makes $O(\log n)$ so cheap at scale.

- **$O(n)$ — linear.** Cost grows in direct proportion to n . A single pass over an array or list.
- **$O(n \log n)$ — linearithmic.** A linear pass repeated $\log n$ times, or $\log n$ work done n times. For example comparison-based sorts (merge sort, heap sort).
- **$O(n^2)$ — quadratic.** Typically a nested loop where the inner loop also runs proportional to n — comparing every element against every other element.
- **$O(2^n)$ — exponential.** Cost doubles with every additional element. Common in naive recursive solutions that branch into two (or more) sub-calls per step, worked through below with Fibonacci.
- **$O(n!)$ — factorial.** Cost multiplies by n with every additional element. Brute-forcing every permutation of n items.

Calculating complexity: practical rules

Sequential blocks add

Two separate loops over two different inputs, one after the other:

```
for (int i = 0; i < n; i++) { /* ... */ } // O(N)
for (int j = 0; j < m; j++) { /* ... */ } // O(M)
```

Total cost is $O(N) + O(M) = O(N + M)$. They add, because they run one after the other, not one inside the other.

Nested blocks multiply

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < m; j++) { /* ... */ }  
}
```

The inner loop runs m times for *every* iteration of the outer loop: $O(N \times M)$.
When both loops run over the same input, this becomes the familiar $O(N^2)$.

Drop constants

```
 $O(2N) \Rightarrow O(N)$   
 $O(N + N) \Rightarrow O(2N) \Rightarrow O(N)$ 
```

Doing the work twice still grows linearly, so the constant factor is dropped.

Drop lower-order terms

```
 $O(N^2 + N) \Rightarrow O(N^2)$ 
```

As n grows large, the N^2 term dominates so completely that the N term becomes irrelevant to the overall shape.

Amortized time

One operation is occasionally expensive but rare enough that it doesn't affect the average.

Example: `ArrayList` (a dynamically resizing array). It gives the benefits of a plain array ($O(1)$ indexed access) plus the ability to grow. Most **`add()`** calls are $O(1)$ — there's spare capacity, so the new element just gets placed at the end. But when capacity runs out, an **`add()`** triggers:

1. Allocate a new backing array (commonly double the size).
2. Copy every existing element into it $\rightarrow O(N)$.
3. Then place the new element.

That single resize is $O(N)$, but it only happens occasionally, and each time it happens the *next* resize is twice as far away. Spread the occasional $O(N)$ cost across all the $O(1)$ inserts that happened since the last resize, and the **average cost per insert is still $O(1)$** — that's amortized $O(1)$, not "sometimes $O(1)$, sometimes $O(N)$ " treated as a worst-case $O(N)$ for every call.

Recursive time complexity

For a recursive function, draw the **call tree** and count the calls.

```
int fibonacci(int n) {  
    if (n ≤ 2) return 1;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Every call that isn't a base case makes two more calls. Call tree for **`fibonacci(5)`**:

```

f(5)
├─ f(4)
│   ├─ f(3)
│   │   ├─ f(2) [base case]
│   │   └─ f(1) [base case]
│   └─ f(2) [base case]
└─ f(3)
    ├─ f(2) [base case]
    └─ f(1) [base case]

```

Time complexity

Each level of the tree can have up to twice as many calls as the level before it:

```

level 0: 1 call
level 1: 2 calls
level 2: 4 calls
...
level n: 2^n calls

```

Add up the levels and the total is roughly $O(2^n)$ — the work doubles with every extra step of `n`.

Space complexity

Space is about the **call stack**: how many calls are waiting at once. `fibonacci` only goes one branch deep at a time, so at most `n` calls stack up. That's $O(n)$ space.

Exercise: Reverse a Singly Linked List

Two ways to solve the same problem, worth comparing directly because they land on the exact same time complexity but a different space complexity.

Problem

Given the head of a singly linked list, reverse it in place and return the new head. Each node only knows its **`next`**, there's no **`prev`** to fall back on:

```
1 → 2 → 3 → 4 → 5 → null
```

becomes

```
5 → 4 → 3 → 2 → 1 → null
```

Tests (RED)

```
@Test
public void shouldReverseIteratively(){
    Day4.ListNode head = fromArray(1, 2, 3, 4, 5);

    Day4.ListNode reversed = Day4.reverseIterative(head);

    assertThat(toArray(reversed)).isEqualTo(new int[]{5, 4, 3, 2, 1});
}
```

```
@Test
public void shouldReverseIterativelyWhenEmpty(){
    assertThat(Day4.reverseIterative(null)).isNull();
}
```

```
@Test
public void shouldReverseIterativelyWhenSingleNode(){
    Day4.ListNode head = fromArray(1);

    Day4.ListNode reversed = Day4.reverseIterative(head);

    assertThat(toArray(reversed)).isEqualTo(new int[]{1});
}
```

Iterative Implementation (GREEN)

```
/*
    reverse iteratively
    O(N) time, O(1) space
*/
public static ListNode reverseIterative(ListNode head) {
    ListNode prev = null;

    while (head != null) {
        ListNode next = head.next;
        head.next = prev;
        prev = head;
        head = next;
    }

    return prev;
}
```

Three pointers do all the work: **`prev`** (what's already reversed), **`head`** (the node currently being processed) and **`next`** (saved before it's overwritten, otherwise the rest of the list is lost the moment **`head.next`** is reassigned). Each node is visited once and each pointer update is $O(1)$, so the loop is **$O(N)$ time**. No extra structure grows with the input, so it's **$O(1)$ space**.

Recursive Implementation

```
/*
    reverse recursively: recurse all the way to the last node first,
    then rewire pointers on the way back up the call stack
    O(N) time, O(N) space (call stack)
*/
public static ListNode reverseRecursive(ListNode head) {
    if (head == null || head.next == null) {
        return head;
    }

    ListNode newHead = reverseRecursive(head.next);

    head.next.next = head;
    head.next = null;

    return newHead;
}
```

The recursion walks forward to the *last* node first — that becomes `newHead`, unchanged all the way back up. Then, as each call returns, it flips one link: `head.next.next = head` makes the next node point back at the current one, and `head.next = null` clears the old forward link (crucial on the original head, otherwise it'd still point forward into the reversed list, forming a two-node cycle).

Same time complexity, different space complexity

Both visit every node exactly once, so both are **O(N) time**. They diverge on space:

- **Iterative** — $O(1)$ space. Three pointers, reused on every iteration, regardless of how long the list is.
- **Recursive** — $O(N)$ space. Every call stays on the call stack waiting for ``reverseRecursive(head.next)`` to return before it can rewire its own pointers.

For a short list the difference doesn't matter. For a very long one, the recursive version risks a ``StackOverflowError`` where the iterative version would run fine.

Entry 5: Data Structures are important

Hash Tables

A data structure for storing key-value pairs so you can look them up fast. On average, look-ups stay $O(1)$ as it grows, because the array resizes once it gets too full.

How are they implemented?

An array, where each slot holds a small linked list instead of a single value:

1. A **hash function** turns the key into a number: `hashCode()` in Java.
2. That number is used to get a valid index: `hashCode % arraySize`.
3. The key-value pair is added to the linked list at that index.

```
key ---> hashCode() ---> hashCode % arraySize ---> linked list at that index
```

Collisions

Two different keys can land on the same index. That's a **collision**. Because each slot is a linked list, not just one value, it simply grows:

```

index    linked list
0        → ["apple", 3] → null
1        → null
2        → ["banana", 7] → ["grape", 1] → null ← collision: two
keys, same slot
3        → ["kiwi", 9] → null

```

Looking up `"grape"` means: hash it to index `2`, then walk that list comparing keys until `"grape"` is found.

Why it's $O(1)$ on average, not always

- **Best case, $O(1)$.** A good hash function spreads keys evenly, so each bucket holds very few entries, often zero or one. Hash it, jump to the bucket, done.
- **Worst case, $O(N)$.** If every key lands in the same bucket, it turns into one long linked list, and lookup becomes a plain linear scan.

Array

- **Fixed size**, set when the array is created.
- Elements sit in **contiguous memory**, so any index can be reached directly:
`address = base + index * elementSize`: $O(1)$ access.
- Inserting or removing in the middle means shifting other elements: $O(N)$.

List (`ArrayList`)

A resizable array, same $O(1)$ indexed access as a plain array, plus growth.

- **$O(1)$ access** by index, same reason as an array (contiguous memory).
- **Amortized $O(1)$ append.** Usually adding an element is instant. When the array is full, every element gets copied into a bigger one, which is $O(N)$. That copy is rare enough that, on average, each insert still costs $O(1)$.
- **$O(N)$ insert/remove in the middle**, everything after the gap has to shift over.

Linked List

A linear structure of nodes, each holding a value and a pointer to the next node (a **doubly** linked list also keeps a pointer to the previous one).

- **No random access.** Reaching the i -th node means walking from the head, one **`next`** at a time. That's $O(N)$.
- **$O(1)$ insert/remove at the head** (and at the tail too, if a tail pointer is kept). Just need to rewire a couple of pointers, nothing shifts.
- **Removing a node you already have a reference to is $O(1)$:** point the previous node's **`next`** straight at the following node. Finding that node in the first place is still $O(N)$.

Stacks

LIFO means Last In, First Out. Think of a stack of plates: you only ever add or take from the top.

- Operations: ``push`` (add to top), ``pop`` (remove from top), ``peek`` (look at the top without removing it), ``isEmpty``.
- All four operations are **O(1)**, but only because they only ever touch the top.

Queues

FIFO means First In, First Out. Think of a line at a store: first to join, first to be served.

- Operations: ``add`` (enqueue, to the back), ``remove`` (dequeue, from the front), ``peek`` (look at the front without removing it), ``isEmpty``.
- Same shape as a stack: ``add``/``remove``/``peek`` are **O(1)** because they only ever touch the two ends.

Exercise: Two Sum

Problem

Write ``indexForTarget(int[] nums, int target)``: return the indices of the two numbers that add up to ``target``. Assume exactly one valid pair exists.

```
nums = [2, 4, 6, 2, 5, 6], target = 11  
→ [2, 4]      (nums[2] + nums[4] = 6 + 5 = 11)
```

Tests (RED)

```
@Test  
public void shouldIndexesForTarget(){  
    Day3 d3 = new Day3();  
  
    assertThat(d3.indexForTarget(new int[]{2,4,6,2,5,6},  
11)).isEqualTo(new int[]{2, 4});  
}
```

Implementation (GREEN)

```
/**
 * one-pass hash map: for each number, check whether its complement
 * (target - number) was already seen before storing the current number
 *
 * O(N) time, O(N) space
 */
public int[] indexForTarget(int[] nums, int target)
{
    Map<Integer, Integer> valueIndex = new HashMap<>();

    for (int i = 0; i < nums.length; i++)
    {
        int diff = target - nums[i];

        if (valueIndex.containsKey(diff))
        {
            return new int[]{valueIndex.get(diff), i};
        }

        valueIndex.put(nums[i], i);
    }

    return new int[]{-1, -1};
}
```

Why a hash map, not nested loops

The brute-force solution uses two nested loops: for every number, scan every other number looking for a complement that sums to **`target`**. That's $O(n^2)$.

```
nums = [2, 4, 6, 2, 5, 6], target = 11
```

```
i=0: need 11-2=9    map has {}                → not found, store 2→0  
i=1: need 11-4=7    map has {2:0}             → not found, store 4→1  
i=2: need 11-6=5    map has {2:0, 4:1}        → not found, store 6→2  
i=3: need 11-2=9    map has {2:0, 4:1, 6:2}    → not found, store 2→3  
(overwrites 2→0)  
i=4: need 11-5=6    map has {2:3, 4:1, 6:2}    → 6 found at index 2 →  
return [2, 4]
```

Entry 6: Graphs and Trees

Used more than you'd think: dependencies, permissions, DB indexes, routing.

Graph

A collection of nodes with edges between them.

- every node can store a list of adjacent vertices
- you could reach potentially each node from any other

```
A --- B --- C
|     |     |
D --- E --- F
```

Adjacency list (each node keeps its own neighbors):

```
A: [B, D]   B: [A, C, E]   C: [B, F]
D: [A, E]   E: [B, D, F]   F: [C, E]
```

Adjacency matrix ($V \times V$ grid, 1 where an edge exists):

	A	B	C	D	E	F
A	[0, 1, 0, 1, 0, 0]					
B	[1, 0, 1, 0, 1, 0]					
C	[0, 1, 0, 0, 0, 1]					
D	[1, 0, 0, 0, 1, 0]					
E	[0, 1, 0, 1, 0, 1]					
F	[0, 0, 1, 0, 1, 0]					

Graph Algorithms

- Depth-First Search $O(V + E)$
- Explore as far as possible on each branch
- pre-order
- boolean "visited" needed
- use it for: does a path exist, cycle detection, topological sort

```

void dfs(Node n){
    Stack stack = new Stack();

    stack.push(n);

    while(!stack.isEmpty()){
        Node stackNode = stack.pop();

        if(!stackNode.visited){
            print(stackNode.data);
            stackNode.visited = true;

            for(Node x: stackNode.adjacents()){
                stack.push(x);
            }
        }
    }
}

```

- Breadth first search $O(V + E)$
- Explore all the vertices before moving to a neighbour node
- use it for: shortest path in an unweighted graph, "degrees of separation"

```

void bfs(Node n){
    Queue<Node> q = new LinkedList<>();

    n.visited = true;
    q.add(n);

    while(!q.isEmpty()){
        Node qNode = q.remove();
        print(qNode.data);

        for(Node x: qNode.adjacents()){
            if(!x.visited()){
                x.visited = true;
                q.add(x);
            }
        }
    }
}

```

Tree

- A connected graph without cycles
- non-linear structure
- organizes data hierarchically

```

      root
     /  \
  child child
   /  \
leaf leaf

```

Binary Tree

- up to two children per node
- a perfect binary tree: every level is filled
- a complete binary tree: every level filled except possibly the last
- balanced: keeps height $O(\log N)$. Combined with the BST rule ($\text{left} < \text{node} < \text{right}$), that's what makes search and inserts $O(\log N)$

```

      4
     / \
    2  6
   / \ / \
  1  3 5  7

```

Going through a binary tree

- in-order

```

void inOrder(TreeNode n){
    if(n != null){
        inOrder(n.left);
        print(n.data);
        inOrder(n.right);
    }
}

```

- pre-order

```

void preOrder(TreeNode n){
    if(n != null){
        print(n.data);
        preOrder(n.left);
        preOrder(n.right);
    }
}

```

- post-order

```

void postOrder(TreeNode n){
    if(n != null){
        postOrder(n.left);
        postOrder(n.right);
        print(n.data);
    }
}

```

- sum all nodes of a binary tree

```
int binTreeSum(TreeNode n){
    if(n == null) return 0;
    return binTreeSum(n.left) + n.data + binTreeSum(n.right);
}
```

Time = $O(N)$: every node visited once, $O(1)$ work per call.

Space = $O(\text{height})$, not flat $O(N)$: only one root-to-leaf path sits on the call stack at a time. $O(\log N)$ balanced, $O(N)$ worst case on a skewed tree, so $O(N)$ isn't wrong as a worst-case bound, just looser than $O(\text{height})$.

Exercise: Balanced Brackets

Problem

Write `isBalancedBrackets(String word)`: given a string that may mix `()[]{}` with other characters, return whether every opening bracket has a matching closing bracket, in the correct order.

"hello(world)"	→ true
"foo[bar{baz}]"	→ true
"hello(world[123]{test})"	→ true
"abc(def[ghi]{jkl})xyz"	→ true
"foo(bar[baz]qux)"	→ false ({ closed by } instead of)

Tests (RED)

```
@Test
public void shouldValidateBalancedBrackets(){
    assertThat(underTest.isBalancedBrackets("hello(world)").isTrue());
    assertThat(underTest.isBalancedBrackets("foo[bar{baz}]").isTrue());
    assertThat(underTest.isBalancedBrackets("hello(world[123]
{test}"))).isTrue();
    assertThat(underTest.isBalancedBrackets("abc(def[ghi
{jkl}]xyz)").isTrue());

    assertThat(underTest.isBalancedBrackets("foo(bar[baz]qux)").isFalse());
}
```

Implementation (GREEN)

```
private Set<Character> openingBrackets = Set.of('(', '{', '[');
private Set<Character> closingBrackets = Set.of(')', '}', ']');

/*
    O(N) time, O(N) space worst case (a string of only opening brackets)
*/
public boolean isBalancedBrackets(String word){
    Stack<Character> opened = new Stack<>();

    for (Character character : word.toCharArray()){

        if (openingBrackets.contains(character)){
            opened.push(character);
        }

        if (closingBrackets.contains(character)){
            if (opened.empty()) return false;

            Character latestOpening = opened.pop();

            if (!isClosingIt(latestOpening, character)) return false;
        }
    }

    return true;
}

private boolean isClosingIt(Character a, Character b){
    return (a.equals('(') && b.equals(')')) ||
           (a.equals('{') && b.equals('}')) ||
           (a.equals '[' && b.equals(']')) ||
}
```

```
(a.equals('(') && b.equals(')'));  
}
```

Why a stack, not a counter

What if we count opens and closes and check they match?

```
"([)]" → 2 opens, 2 closes, "balanced" by count, but not actually  
valid
```

- That breaks immediately on ordering.

A stack fixes this because it remembers *which* bracket is still waiting to be closed, not just how many. Every closing bracket must match the *most recently opened, still-unclosed* bracket, and that's exactly what `pop()` gives you: LIFO, last in, first out.

Walking `"([)]"` through the stack:

```
'(' → push      stack: [(  
'[' → push      stack: [( [  
)' → pop '['    '[' vs ')' don't match → false, bail out immediately
```

Entry 6 extra: Crazy extra bits and StringBuilder

Bit Manipulation

Operations

- `<<` left shift: shift bits left, fill with 0 on the right.
- `>>` right shift (signed): shift bits right, fill on the left with the **sign bit** (0 for positive, 1 for negative). Preserves the sign, same as dividing by 2 per shift (rounded toward negative infinity).
- `>>>` unsigned right shift: shift bits right, always fill on the left with 0, ignoring the sign. Only differs from `>>` on negative numbers.

```
5 >> 1 = 2      0000...0101 >> 1 = 0000...0010
-8 >> 1 = -4     1111...1000 >> 1 = 1111...1100  (sign bit carried
in)
-8 >>> 1 = big + 1111...1000 >>> 1 = 0111...1100  (0 carried in, sign
lost)
```

Truth table

A	B	A & B (AND)	A B (OR)	A ^ B (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

`~` (NOT) is unary, it flips a single bit:

A	~A
0	1
1	0

- `&` is 1 only if **both** bits are 1.
- `|` is 1 if **at least one** bit is 1.
- `^` is 1 if the bits are **different**.

Magic Tricks

Each trick below builds on one operator from the truth table above.

Clear a bit: `num & ~mask`

`~(1 << pos)` is all 1s except a 0 at `pos`. When an AND is applied with it forces that position to 0 and leaves every other bit untouched (`x & 1 = x`, `x & 0 = 0`).

```
num = 0110 (6), pos = 1
mask = ~(1 << 1) = ~0010 = 1101
num & mask = 0110 & 1101 = 0100 (4, bit 1 cleared)
```

```
// O(1) time, O(1) space
int clearBit(int num, int pos){
    int mask = ~(1 << pos);
    return (mask & num);
}
```

Toggle a bit: `num ^ mask`

`1 << pos` is all 0s with a single 1 at `pos`. When XOR is applied it flips only that bit: `x ^ 1` flips `x`, `x ^ 0` leaves `x` alone.

```
num = 0110 (6), pos = 0
mask = 1 << 0 = 0001
num ^ mask = 0110 ^ 0001 = 0111 (7, bit 0 turned on)

num = 0111 (7), pos = 0
num ^ mask = 0111 ^ 0001 = 0110 (6, bit 0 turned off again)
```

```
// O(1) time, O(1) space
int toggleBit(int num, int pos){
    int mask = 1 << pos;
    return (num ^ mask);
}
```

Appear only once

- Given an array where every number appears twice except one, find the one that appears only once.

Two facts about XOR make this work: $x \oplus x = 0$ (a number cancels itself out) and $x \oplus 0 = x$ (XOR with 0 is a no-op), and XOR doesn't care about order. XOR the whole array together and every pair cancels out, leaving only the number with no partner.

```
data = [4, 1, 2, 1, 2]

result = 0
result ^= 4 → 4
result ^= 1 → 5
result ^= 2 → 7
result ^= 1 → 6    (the two 1s have now cancelled: 5^1^1 = 5)
result ^= 2 → 4    (the two 2s have now cancelled)
```

```
// O(N) time, O(1) space
int appearOnce(int[] data){
    int result = 0;

    for(int num: data){
        result ^= num;
    }

    return result;
}
```

Check a bit: `(num & mask) != 0`

`1 << pos` isolates that one position. When AND is applied with `num` it zeroes out every other bit, so the result is non-zero only if `num` had a 1 there.

```
// O(1) time, O(1) space
boolean isPositionOn(int num, int pos){
    int mask = 1 << pos;
    return (num & mask) != 0;
}
```

Set a bit: `num | mask`

`1 << pos` is a single 1 at `pos`. When OR is applied it forces that position to 1 and leaves every other bit untouched (`x | 0 = x`).

```
// O(1) time, O(1) space
int setBit(int num, int pos){
    int mask = 1 << pos;
    return (num | mask);
}
```

Why we need StringBuilder?

```
String joinWords(String[] manyWords){
    String mySentence = "";

    for(String word: manyWords){
        mySentence += word;
    }
    return mySentence;
}
```

``String`` is immutable. Every iteration allocates a brand new ``String`` and copies every character seen so far into it, plus the new word.

Let ``x`` be how many characters are in one word (assume every word is the same length, to keep the numbers simple), and ``N`` the number of words.

Step 1: from the loop to a series

Before the loop starts, ``mySentence`` is empty. Walk through what each iteration actually has to copy:

```
before iteration 1: mySentence holds 0 words = 0 chars
  += word1 → copy the 0 existing chars, then the x new ones ⇒ 1x
chars copied

before iteration 2: mySentence holds 1 word = 1x chars
  += word2 → copy the 1x existing chars, then the x new ones ⇒ 2x
chars copied

before iteration 3: mySentence holds 2 words = 2x chars
  += word3 → copy the 2x existing chars, then the x new ones ⇒ 3x
chars copied

...

before iteration N: mySentence holds (N-1) words = (N-1)x chars
  += wordN → copy the (N-1)x existing chars, then the x new ones ⇒
Nx chars copied
```

Each iteration `i` copies `i * x` characters, because that's how much of the sentence has piled up so far. Add every iteration's cost together and that's the *total* work the whole loop does:

```
total copies = 1x + 2x + 3x + ... + Nx
```

That's the series: it's not something we chose, it just falls out of tracing what the loop does on each pass.

Step 2: from the series to $O(N^2)$

`x` is the same in every term, so factor it out:

$$1x + 2x + 3x + \dots + Nx = x * (1 + 2 + 3 + \dots + N)$$

`1 + 2 + 3 + ... + N` is the sum of the first `N` natural numbers. There's a well-known shortcut for it (pair the smallest with the largest: `**1+N**`, `**2+(N-1)**`, `**3+(N-2)**`, ... every pair adds up to `**N+1**`, and there are `**N/2**` such pairs):

$$1 + 2 + 3 + \dots + N = N(N+1)/2$$

Substitute that back in:

$$x * (1 + 2 + \dots + N) = x * N(N+1)/2 = x * (N^2 + N) / 2$$

Now apply the usual Big O simplification rules: **drop the constant factor** (`x` and `**1/2**` don't grow with `N`, so they don't matter at scale) and **drop the lower-order term** (`N` is insignificant next to `**N^2**` once `N` is large):

$$x * (N^2 + N) / 2 \longrightarrow O(N^2)$$

StringBuilder Solution O(N)

`**StringBuilder**` is mutable: it keeps a resizable internal `**char[]**` buffer and `**append()**` writes into it directly (amortized O(1) per call, same idea as `**ArrayList**` growth), no copying the whole string built so far.

```
// O(N) time: one append per word, no re-copying on each iteration
String joinWords(String[] words){
    StringBuilder sb = new StringBuilder();

    for(String w: words){
        sb.append(w);
    }

    return sb.toString();
}
```

Exercise: Rotate an Array

Problem

Write `rotateArray(int[] arr, int k)`: rotate the array in place so the last `k` elements move to the front. No new array, $O(1)$ extra space.

```
arr = [1, 2, 3, 4, 5], k = 2
→ [4, 5, 1, 2, 3]
```

Tests (RED)

```
@Test
public void shouldRotateKElements(){
    int[] data = new int[]{1, 2, 3, 4, 5};

    Day4.rotateArray(data, 2);

    assertThat(data).isEqualTo(new int[]{4, 5, 1, 2, 3});
}
```

Implementation (GREEN)

```
/**
 * reversal trick: reverse whole array, then reverse [0, k-1], then
reverse [k, n-1]
 * each element is touched a constant number of times across the three
passes, in place
 *
 * O(n) time, O(1) space
 */
public static void rotateArray(int[] arr, int k){
    rotateSubarray(arr, 0, arr.length - 1);
    rotateSubarray(arr, 0, k - 1);
    rotateSubarray(arr, k, arr.length - 1);
}

public static void rotateSubarray(int[] arr, int start, int end){
    while (start < end){

        int aux = arr[start];
        arr[start] = arr[end];
        arr[end] = aux;

        start++;
        end--;
    }
}
```

Why reverse three times, not shift k times

The obvious approach is to shift every element right by one, k times. That's $O(n \cdot k)$.

Reversing the whole array first puts every element in exactly the reverse of where it needs to end up. From there, the front k and the back $n-k$ are each internally backwards, but already in the right *halves*. Reversing each half back to front fixes that, all in place:

```
arr = [1, 2, 3, 4, 5], k = 2
```

```
reverse all:      [5, 4, 3, 2, 1]
```

```
reverse [0, k-1]: [4, 5, 3, 2, 1]   (fixes the front 2)
```

```
reverse [k, n-1]: [4, 5, 1, 2, 3]   (fixes the back 3)
```

Three passes, each one $O(n)$ at worst, so the total is still $O(n)$

Entry 7: System Design concepts

CAP Theorem

- It is not possible for a distributed system to simultaneously provide consistency, availability and partition tolerance. Only 2 can be picked.
- In practice networks partition whether you want it or not, so the real choice is: during a partition, favor consistency or favor availability.

Distributed Systems

- A collection of independent nodes working together, over a network, to achieve the same goal.

System Design

Thinking about scalability could save time and money.

Scalability

The capability of a system to grow and handle an increasing demand without losing performance. A scalable system balances the load between all the participating nodes.

- Horizontal scaling: adding more nodes into your pool of resources
- Vertical scaling: adding more CPU, RAM or storage to an existing server.

Reliability

In case of failure the system will remain operative. It will keep delivering its services even when some of the components are failing.

- Redundancy and Replication Strategies

Redundancy: duplication of nodes, removes a single point of failure. Replication: copying data across multiple nodes to keep them in sync (e.g. leader/follower).

Availability

It is the percentage of time that a system is operative, including through failures. An AP system sacrifices consistency to stay responsive as much as possible.

Consistency Patterns

- The data must be the same across the system. All the users get the same data at the same time independently of the node.

Eventual consistency

- The inconsistency is temporary and events are eventually synchronized everywhere.

Strong consistency

- The data synchronization is a priority. Every read operation must always reflect the latest value.

Causal consistency

- Operations that are causally related (one depends on the other) must be seen in the same order by everyone. Unrelated (concurrent) operations are free to appear in a different order for different observers.

Partition Tolerance

The system keeps working even when the network between nodes breaks down and some nodes can't talk to each other, not just when a node is down.

Reliability vs Availability

IF IT IS RELIABLE => IT IS AVAILABLE but being AVAILABLE does not mean it is RELIABLE.

Other concepts

Idempotency

A repeated operation will always produce the same result, no matter how many times it runs.

Efficiency

- Response time: it is the latency. Low latency => highly responsive system.
- Throughput: the rate of data or operations a system processes per unit of time, e.g. requests/sec or bytes/sec.

Monolith vs Microservices

Monolith

- A single codebase and deployable unit.
- Coupling components
- single point of failure

Microservices

- Independent self contained services
- easier to scale and deploy
- can use multiple technologies
- increase complexity
- network latency

Exercise: Permutation Check

Problem

Write `isApermutation(String a, String b)`: return `true` if `b` is a permutation of `a`, meaning both strings have exactly the same characters, in any order (and the same length).

```
isApermutation("ab", "ba") → true
isApermutation("ab", "ava") → false    (different length)
isApermutation("ab", "aa") → false    (same length, different
characters)
```

Tests (RED)

```
@Test
public void shouldCheckPermutations(){
    Day2 day2 = new Day2();

    assertThat(day2.isApermutation("ab", "ba")).isTrue();
    assertThat(day2.isApermutation("ab", "ava")).isFalse();
    assertThat(day2.isApermutation("ab", "aa")).isFalse();
}
```

Implementation (GREEN)

```
/**
 * count every character of a into a fixed-size (128, ASCII) *frequency
 * table, then walk b decrementing counts. If a count *goes negative, b has
 * a character a doesn't (or has it too *many times), so it can't be a
 * permutation.
 *
 * *O(N) time, O(1) space (the frequency table is capped at 128
 * *regardless of N)
 */
public boolean isApermutation(String a, String b)
{
    if(a.length()  $\neq$  b.length()) return false;

    int[] freqA = freqTable(a);
    char[] charsB = b.toCharArray();

    for (int i = 0; i < charsB.length; i++)
    {
        if (freqA[charsB[i]]  $\geq$  0) freqA[charsB[i]]--;

        if (freqA[charsB[i]] < 0) return false;
    }

    return true;
}
```

Why a frequency table, not sorting

The obvious approach is to sort both strings and compare them character by character, since two strings with the same characters become identical once

sorted. That works, but sorting costs $O(n \log n)$.

Counting instead of sorting drops that to $O(n)$: build a frequency table for `a`, then walk `b` once, decrementing as you go. If `b` is a true permutation, every count lands back at exactly 0.

```
a = "ab", b = "ba"

freqTable(a):      {a: 1, b: 1}

walk b:
  'b' → freqA[b] 1 → 0
  'a' → freqA[a] 1 → 0

no count went negative → true
```

```
a = "ab", b = "aa"

freqTable(a):      {a: 1, b: 1}

walk b:
  'a' → freqA[a] 1 → 0
  'a' → freqA[a] 0 → -1  (already at 0, 'a' appears once too often)

count went negative → false
```

Entry 8: Load Balancers and Caching

Load Balancing

Helps spread traffic across a cluster of servers, improving response time and availability.

- Sits between the client and the server
- Reduces individual server load and prevents a single point of failure

Basic topology

```
clients → [ load balancer ] --+→ server 1
                                +→ server 2
                                +→ server 3
```

Benefits

- faster, uninterrupted experience
- less downtime
- higher throughput can be handled

Algorithms

- round robin: requests distributed sequentially across servers. The current load is ignored
- weighted round robin: same idea, but servers with more capacity get a larger share
- least connections: routes to the server with the fewest active connections
- least response time: like least connections, but also factors in current average response time

Failover patterns

- active-passive: one node handles traffic, a standby node takes over only if it fails (detected via health checks)
- active-active: multiple nodes handle traffic at the same time; if one fails, the others absorb its load

Caching

Improves performance by storing frequently accessed data in temporary storage

- recently requested data is likely to be requested again (temporal locality)
- short-term memory: limited amount of space but faster than the original source of data

Content delivery network (CDN)

A geographically distributed set of cache servers that serve content from a location close to the client.

- useful for sites serving large amounts of static content

Cache writing policies

How a write reaches the cache and the backing store.

- write-through: cache and db updated at the same time, strong consistency but higher latency
- write-around: writes go directly to the db, skipping the cache; the cache is only populated on the next read.
- write-back (write-behind): write the cache only, the database is synchronized later, asynchronously. low latency but risk of losing data if the cache fails.

Cache invalidation

If the data changed in the database, stale entries already sitting in the cache need to go, or be refreshed.

- TTL (time-to-live): each entry expires automatically after a set duration, stale data can still be served until then
- explicit invalidation: the write path purges (or updates) the matching cache entry as part of the write, tighter consistency but couples the write path to the cache

Eviction

Removing stored data from the cache when running out of space.

- FIFO (first-in-first-out)
- LIFO (last-in-first-out)
- LRU (least-recently-used)
- LFU (least-frequently-used)

Exercise: URLify

Problem

Write `urlSafeString(String word)`: replace every space in a string with `%20`, trimming leading/trailing whitespace first.

```
"Mr John Smith" → "Mr%20John%20Smith"  
"a  b"          → "a%20%20b"  
"Hello"         → "Hello"
```

Tests (RED)

```
@Test
public void shouldMakeStringsURLSafe(){
    assertThat(underTest.urlSafeString("Mr John Smith"))
        .isEqualTo("Mr%20John%20Smith");

    assertThat(underTest.urlSafeString("Hello World"))
        .isEqualTo("Hello%20World");

    assertThat(underTest.urlSafeString("Java Is Fun"))
        .isEqualTo("Java%20Is%20Fun");

    assertThat(underTest.urlSafeString("a b"))
        .isEqualTo("a%20%20b");

    assertThat(underTest.urlSafeString("Hello"))
        .isEqualTo("Hello");
}
```

Implementation (GREEN)

```
/*
    n = trimmed length, s = number of spaces
    totalSpaces: one pass, O(n)
    main loop: one pass, O(n)
    O(n) time, O(n) space (result array is n + 2s chars)
*/
public String urlSafeString(String word){
    char[] charArray = word.trim().toCharArray();
    char[] charArrayResult = new char[charArray.length +
(totalSpaces(word.trim()) * 2)];

    int index = charArrayResult.length;

    for (int i = charArray.length - 1; i ≥ 0; i--){
        if (Character.isWhitespace(charArray[i])){
            charArrayResult[--index] = '0';
            charArrayResult[--index] = '2';
            charArrayResult[--index] = '%';
        }
        else {
            charArrayResult[--index] = charArray[i];
        }
    }

    return new String(charArrayResult);
}

private int totalSpaces(String word){
    int result = 0;

    for (int i = 0; i < word.length(); i++){
        if (Character.isWhitespace(word.charAt(i)))
```

```
        result++;  
    }  
    return result;  
}
```

Why fill from the back

Each space grows from 1 character to 3 (``%20``). Filling left to right would keep pushing later characters further along, forcing re-shifts, $O(n \cdot s)$ worst case. Filling right to left instead: the final length is already known, so every character has one fixed slot and gets written exactly once. $O(n)$ total.

Entry 9: Scaling the Database

Sharding / Data Partitioning

Splitting a single database into smaller pieces spread across multiple machines, so no one node has to store or serve all the data.

Horizontal Partitioning (Sharding)

Rows are the unit of split. The same table is cut into pieces called shards, each holding a subset of the rows on its own machine.

```
Locations table
+-----+-----+-----+
| zip < 33k | 33k-66k | zip ≥ 66k |
| shard 1   | shard 2   | shard 3   |
+-----+-----+-----+
```

Vertical Partitioning

Columns are the unit of split. A single row's fields live on different machines.

users	
+-----+	+-----+
profile columns	metadata columns
(name, email)	(last_login...)
shard 1	shard 2
+-----+	+-----+

Example: ``user.profile`` is read on every page load, ``user.metadata`` rarely is. Splitting them keeps hot and cold data from competing for the same cache and disk.

Partitioning Strategies

These strategies decide which shard a given row lands in. They only apply to horizontal partitioning; vertical partitioning is a fixed decision made at table design time, not a per-row one.

Range-based

Partitions are assigned a contiguous range of a chosen key.

```
zip_code < 33000      → shard 1
33000 ≤ zip_code < 66000 → shard 2
zip_code ≥ 66000     → shard 3
```

Simple, and range queries ("all zips between X and Y") stay on one or two shards. Downside: an uneven key distribution creates hot shards, for example most rows clustering just above one of the boundaries.

Hash-based

A hash function maps the entity's key to a shard number.

```
shard = hash(id) % number_of_shards
```

Spreads data evenly regardless of the key's natural distribution. The problem is resharding: adding or removing a shard changes the divisor, so `hash(id) % number_of_shards` gives a different result for almost every key, and nearly all the data has to move. Consistent hashing is the usual fix, it remaps only a small slice of keys when the shard count changes.

List-based

Each partition is assigned an explicit list of values.

```
shard 1: Europe  
shard 2: Americas  
shard 3: Asia
```

Good when the partitioning key already has meaningful, known groupings, like region, tenant, or business unit.

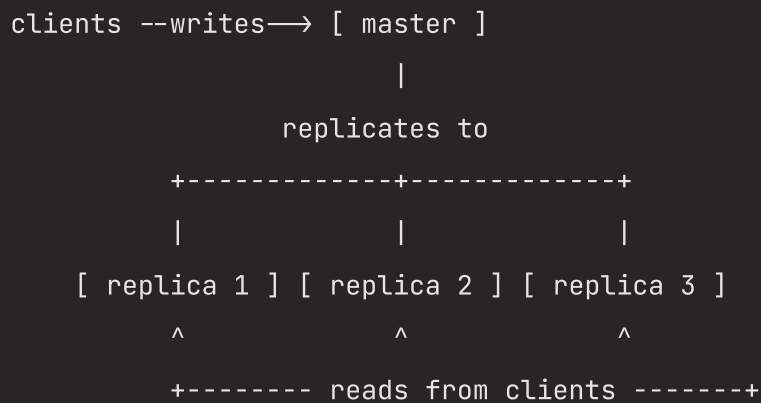
Partitioning Drawbacks

- Data no longer lives on one server, so a query touching multiple shards has to be sent to each one separately, then the results combined by hand.

- Joins across shards aren't supported by the database anymore. They have to be done in application code, or avoided by design.
- Referential integrity (foreign keys) can't be enforced across shards by the database. It becomes the application's responsibility.

Database Replication

Copying the same data across multiple nodes, primarily to scale *read* operations and to survive a node going down.



- **Master:** handles write operations, the single source of truth.
- **Replicas (slaves):** handle read operations, scaling read capacity horizontally.
- If the master fails, one replica is promoted to become the new master (failover), the same idea as active-passive load balancing.

Synchronous vs asynchronous replication

- **Synchronous:** the master waits for at least one replica to confirm the write before acknowledging it to the client. Strong consistency, higher write

latency.

- **Asynchronous:** the master acknowledges the write immediately and replicates in the background. Low latency, but a replica can briefly serve stale data, and a write can be lost if the master crashes before replicating.

Indexes

A separate data structure, built on one or more columns of a table, that lets the database find matching rows without scanning every row.

```
index (on last_name)           table
+-----+                     +-----+
| "Adams" → row 42 |----->| row 42: Adams... |
| "Diaz"  → row 7  |----->| row 7:  Diaz...  |
| "Osei"  → row 15 |----->| row 15: Osei...  |
+-----+                     +-----+
```

- Usually built as a **B-tree**, a sorted structure that makes lookups fast: `O(log n)`.
- Speeds up `WHERE`, `JOIN`, and `ORDER BY` on the indexed column(s).
- Trade-off: every `INSERT`/`UPDATE`/`DELETE` has to update the index too, and the index itself takes extra storage.

Exercise: Min Stack

Problem

Design a stack that supports `push`, `pop`, `top`, and `getMin`, all in $O(1)$ time, `getMin` included.

```
push(5)  → stack: [5]           getMin() → 5
push(3)  → stack: [5, 3]        getMin() → 3
push(7)  → stack: [5, 3, 7]     getMin() → 3
pop()    → removes 7            getMin() → 3
pop()    → removes 3            getMin() → 5
```

Tests (RED)

```
@Test
public void shouldTrackMinAsElementsArePushed(){
    underTest.push(5);
    assertThat(underTest.getMin()).isEqualTo(5);

    underTest.push(3);
    assertThat(underTest.getMin()).isEqualTo(3);

    underTest.push(7);
    assertThat(underTest.getMin()).isEqualTo(3);
}

@Test
public void shouldRestorePreviousMinAfterPoppingDuplicateMin(){
    underTest.push(5);
    underTest.push(3);
    underTest.push(3);
    underTest.push(7);

    underTest.pop(); // removes 7
    assertThat(underTest.getMin()).isEqualTo(3);

    underTest.pop(); // removes one of the 3s, other 3 still there
    assertThat(underTest.getMin()).isEqualTo(3);

    underTest.pop(); // removes last 3
    assertThat(underTest.getMin()).isEqualTo(5);
}

@Test
public void shouldReturnTopWithoutRemovingIt(){
    underTest.push(5);
```

```
    underTest.push(9);

    assertThat(underTest.top()).isEqualTo(9);
    assertThat(underTest.top()).isEqualTo(9); // unchanged, top()
    doesn't pop
}

@Test
public void shouldThrowOnEmptyStackOperations(){
    assertThatThrownBy(() →
underTest.pop()).isInstanceOf(EmptyStackException.class);
    assertThatThrownBy(() →
underTest.top()).isInstanceOf(EmptyStackException.class);
    assertThatThrownBy(() →
underTest.getMin()).isInstanceOf(EmptyStackException.class);
}
```

Implementation (GREEN)

```
private Stack<Integer> values = new Stack<>();
private Stack<Integer> mins = new Stack<>();

/**
 * O(1) time, O(1) extra space per call (amortized O(n) total across n
 * pushes)
 */
public void push(int value) {
    values.push(value);
    mins.push(mins.isEmpty() ? value : Math.min(value, mins.peek()));
}

/**
 * O(1) time
 */
public int pop() {
    if (values.isEmpty()) throw new EmptyStackException();

    mins.pop();
    return values.pop();
}

/**
 * O(1) time
 */
public int top() {
    if (values.isEmpty()) throw new EmptyStackException();

    return values.peek();
}

/**
```

```

* 0(1) time
*/
public int getMin() {
    if (mins.isEmpty()) throw new EmptyStackException();

    return mins.peek();
}

```

Why a second stack, not a single "current min" field

The obvious first attempt: keep one `min` variable, update it on every push.

```

push(5) → min = 5
push(3) → min = 3
push(7) → min = 3    (unchanged)
pop()   → removes 7, min still 3, fine
pop()   → removes 3... but what was the min before 3 was pushed?

```

That's the break: a single variable only remembers the *current* minimum, not the history of minimums. Once the value that set it gets popped, there's no way back to the previous one short of rescanning the whole stack, `O(n)`, when `getMin` is supposed to be `O(1)`.

The fix mirrors the values stack itself, tracking one min per push:

```
push(5) → values: [5]      mins: [5]
push(3) → values: [5,3]    mins: [5,3]
push(3) → values: [5,3,3]  mins: [5,3,3]
push(7) → values: [5,3,3,7] mins: [5,3,3,7] (7 > 3, min unchanged)
pop()   → values: [5,3,3]  mins: [5,3,3] (drop 7's min too)
pop()   → values: [5,3]    mins: [5,3]   (min correctly back to 3)
```

No rescans, no lost history.

Entry 10: System Design Concepts Mix

SQL vs NoSQL

Relational databases are structured, with a fixed, predefined schema.

- **SQL:** data organized into rows and columns, fixed schema.
- **ACID compliant:** reliability, every transaction leaves the data in a consistent state.

Non-relational databases are unstructured, with a dynamic schema that can vary.

- **Key-value store** (Redis): every record is a key mapped to a value, nothing more.
- **Document-based:** data grouped into collections, but each document in a collection can have a different structure.
- **Horizontal scalability:** easier to scale by adding more machines.
- Trades ACID guarantees for performance and scalability.

Reasons to use SQL

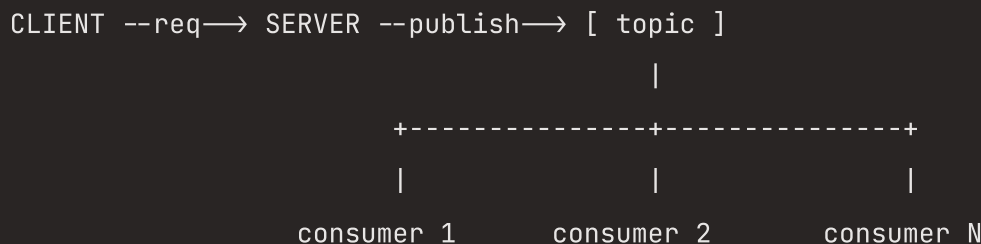
- ACID compliance is required.
- The data is structured and its shape is known upfront.

Reasons to use NoSQL

- Volume of data would otherwise become the bottleneck.
- Large volumes of data need to be stored and the structure isn't important, or varies too much to fit a fixed schema.

Pub/Sub

A producer publishes messages to a topic instead of talking to consumers directly. Every consumer subscribed to that topic gets its own copy of the message.



Kafka is the usual example.

- Decouples producers from consumers, neither needs to know the other exists.
- Messages are persisted, a consumer that comes back online can catch up.
- Easy to scale: add more consumers to spread the processing load.
- Multiple consumers can read the same message independently.

Disadvantages

- **No atomicity:** publishing several messages isn't a single unit of work, some can succeed while others fail.
- **No idempotency:** delivery is usually at-least-once, so a consumer can see the same message twice (e.g. after a retry) and has to be written to handle duplicates itself.

Message queue, not the same thing

A **message queue** (RabbitMQ) looks similar but the delivery model is different: each message is consumed by exactly one consumer, not every subscriber. Multiple consumers reading the same queue compete for messages, which spreads load rather than fanning the same message out to everyone. It's middleware used to let services communicate asynchronously without a topic's broadcast behavior.

API Gateway

A single entry point that sits in front of multiple backend services.

- Every client talks to the gateway, not to the services directly.
- Routes each request to the right backend service.
- Common place to handle cross-cutting concerns once instead of in every service: authentication, rate limiting, and logging.

Proxy

An intermediate server between the client and the backend that can filter, transform, or forward a request.

- A **forward proxy** sits in front of the client, hiding the client's identity from whatever server it talks to.
- A **reverse proxy** sits in front of the backend instead, hiding the backend's servers from the client. It can also terminate SSL, cache responses, or compress them before they reach the client.

Load Balancer vs API Gateway

Both sit between the client and "the backend", which is what makes them easy to mix up with each other, and with a reverse proxy. A **load balancer** is really a reverse proxy specialized for one job: spreading traffic across many *replicas of the same service*. An **API gateway** routes to many *different services* instead of replicas of one, and adds concerns that apply across all of them: auth, rate limiting, request/response transformation.

```
Load balancer:  client → [ LB ] → service A (replica 1)
                                   → service A (replica 2)
                                   → service A (replica 3)
```

```
API gateway:   client → [ GW ] → users service
                                   → orders service
                                   → payments service
```

Exercise: Thread-Safe Counter

Problem

A simple `long value; value++;` counter is unsafe under concurrent writes: `++` isn't a single CPU instruction, it's read-modify-write, so two threads can read the same value, both increment it, and one increment gets lost.

```
20 threads, 10,000 increments each → expected total: 200,000
```

unsafe long:	< 200,000	(lost updates)
synchronized:	200,000	(correct, but threads queue up)
AtomicLong:	200,000	(correct, no queuing)

Tests (RED)

```
private static final int THREAD_COUNT = 20;
private static final int INCREMENTS_PER_THREAD = 10_000;

@Test
public void syncCounterShouldCountAllIncrementsAcrossThreads() throws
InterruptedException {
    SynchCounter underTest = new SynchCounter();

    runConcurrently(underTest::increment);

    assertThat(underTest.getValue()).isEqualTo((long) THREAD_COUNT *
INCREMENTS_PER_THREAD);
}

@Test
public void atomicCounterShouldCountAllIncrementsAcrossThreads() throws
InterruptedException {
    AtomicSafeCounter underTest = new AtomicSafeCounter();
    runConcurrently(underTest::increment);
    assertThat(underTest.getValue()).isEqualTo((long) THREAD_COUNT *
INCREMENTS_PER_THREAD);
}

private void runConcurrently(Runnable increment) throws
InterruptedException {
    ExecutorService pool = Executors.newFixedThreadPool(THREAD_COUNT);
    for (int i = 0; i < THREAD_COUNT; i++) {
        pool.submit(() → {
            for (int j = 0; j < INCREMENTS_PER_THREAD; j++) {
                increment.run();
            }
        });
    }
}
```

```
    }  
    pool.shutdown();  
    pool.awaitTermination(10, TimeUnit.SECONDS);  
}
```

Both implementations share one interface:

```
public interface ThreadSafeCounter {  
    void increment();  
    long getValue();  
}
```

Implementation (GREEN)

synchronized

```

public class SynchCounter implements ThreadSafeCounter {

    private long value = 0;

    @Override
    public void increment() {
        inc();
    }

    @Override
    public long getValue() {
        return get();
    }

    private synchronized void inc() {
        value++;
    }

    private synchronized long get() {
        return value;
    }
}

```

`synchronized` puts a lock around `value`. Only one thread can be inside `inc()` or `get()` at a time, every other thread trying to enter has to wait for the lock to free up.

AtomicLong

```
public class AtomicSafeCounter implements ThreadSafeCounter {

    private final AtomicLong value = new AtomicLong(0);

    @Override
    public void increment() {
        value.incrementAndGet();
    }

    @Override
    public long getValue() {
        return value.get();
    }
}
```

No lock. `incrementAndGet()` is built on a CPU-level compare-and-swap (CAS): read the current value, compute the new one, then write it back only if nobody else changed it in between. If another thread got there first, retry with the fresh value. No thread ever blocks, it just occasionally redoes a bit of work.

Entry 11: Spring Roots

Spring

An open-source framework that provides the context to execute Java applications: it manages "beans" (the objects your code doesn't have to construct by hand), configuration, and other resources. It's layered and modular. A project only pulls in the pieces it needs.

Core modules:

- **core:** the IoC container itself, the foundation everything else sits on
- **data access:** JDBC, transaction management, ORM integration
- **web:** MVC, REST, the servlet layer
- **test:** support for writing unit and integration tests

Inversion of Control (IoC)

The principle behind all of this. A long time ago we had to instantiate what we needed: `new UserRepository()`. Spring flips that: the framework creates the objects and hands them to your code instead. Control over *when* and *how* dependencies get built moves out of your class and into the container. That's the "inversion".

This makes it much easier to test our components in isolation by mocking the dependencies.

Dependency Injection (DI)

DI is how Spring actually achieves IoC: a class declares what it needs, and something outside the class provides it, instead of the class constructing its own dependencies.

- **Constructor injection:** dependencies passed in through the constructor. The preferred style, fields can be `final`, and the class can be tested with plain `new` and mocks, no Spring container required.
- **Setter injection:** dependencies set through a setter after construction. Useful for optional dependencies that aren't required to build a valid object.
- **Field injection** (`@Autowired` directly on a field): shortest to write, but the class can't be constructed without reflection or a running container, which makes plain unit tests harder. Generally avoided in favor of constructor injection.

The IoC Container

The container, usually accessed as an `ApplicationContext`, is what performs the wiring described above.

@Component classes	IoC Container
+-----+	+-----+
UserService	1. scan classpath, find beans
needs UserRepo	2. resolve each dependency
UserRepository	3. inject (wire) them together
+-----+	4. manage scope + lifecycle
	+-----+

- creates and wires objects (beans)
- manages each bean's lifecycle: instantiate, populate its dependencies, run any init callbacks, keep it ready for use, run destroy callbacks on shutdown
- bean scopes: **singleton** (default, one shared instance per container), **prototype** (a new instance every time the bean is requested), **request** / **session** (web-only, one instance per HTTP request or session)
- autowiring: the container matches a dependency by type (or name) to a bean it already manages, and injects it, no manual wiring code needed

Stereotype annotations

The annotations that mark a class as a bean the container should manage:

- `@Component`: generic bean, the base annotation the others build on
- `@Service`: business logic layer
- `@Repository`: persistence layer
- `@Controller` / `@RestController`: web layer. `@RestController` is `@Controller` plus `@ResponseBody`, the return value is serialized straight to the response body (JSON) instead of resolved to a view.

Spring Boot

Takes an opinionated view of Spring: instead of wiring everything by hand, it configures sensible defaults based on the starters (dependencies) added to the project.

- no XML configuration
- faster path to production, convention over manual setup
- embedded HTTP server (Tomcat by default), no separate servlet container to deploy the artifact

Main annotations

`@SpringBootApplication` is shorthand for three annotations combined:

- `@Configuration`: marks the class as a source of bean definitions
- `@ComponentScan`: scans the package (and subpackages) for `@Component`-annotated classes to register as beans
- `@EnableAutoConfiguration`: configures beans automatically based on what's on the classpath, for example a `DataSource` bean appears once a JDBC driver dependency is present

`@RestController`: handles incoming requests, see stereotype annotations above.

`@SpringBootTest`: loads the full application context, for integration tests that need real wiring instead of mocks.

Actuator

Adds production-ready endpoints for inspecting a running application without attaching a debugger.

- `/health`: whether the app and its dependencies (database, disk space...) are up
- `/metrics`: runtime metrics, JVM memory, request counts, and similar
- useful for real-time monitoring, and for catching a performance regression before it becomes an incident

Exercise: LRU Cache

LRU eviction strategy

Problem

Design a cache with a fixed capacity that supports `get` and `put`, both in $O(1)$ time. When `put` is called at full capacity, evict the least-recently-used entry first. A `get` call counts as "using" that entry.

```
capacity 2
put(1, 100)      → cache: [1]
put(2, 200)      → cache: [1, 2]
get(1)           → 100, and 1 is now the most recently used
put(3, 300)      → capacity exceeded, evicts 2 (least recently
used), not 1
get(2)           → -1, evicted
```

Tests (RED)

```
private LRUCache underTest = new LRUCache(2);

@Test
public void shouldReturnMinusOneForMissingKey(){
    assertThat(underTest.get(1)).isEqualTo(-1);
}

@Test
public void shouldReturnValueJustPut(){
    underTest.put(1, 100);

    assertThat(underTest.get(1)).isEqualTo(100);
}

@Test
public void shouldUpdateValueForExistingKeyWithoutEvicting(){
    underTest.put(1, 100);
    underTest.put(2, 200);
    underTest.put(1, 111); // update, not a new key

    assertThat(underTest.get(1)).isEqualTo(111);
    assertThat(underTest.get(2)).isEqualTo(200); // still there
}

@Test
public void shouldEvictLeastRecentlyUsedWhenCapacityExceeded(){
    underTest.put(1, 100);
    underTest.put(2, 200);
    underTest.put(3, 300); // capacity 2, evicts key 1

    assertThat(underTest.get(1)).isEqualTo(-1);
    assertThat(underTest.get(2)).isEqualTo(200);
}
```

```
        assertThat(underTest.get(3)).isEqualTo(300);
    }

    @Test
    public void shouldTreatGetAsUsageProtectingFromEviction(){
        underTest.put(1, 100);
        underTest.put(2, 200);
        underTest.get(1); // key 1 is now most-recently-used
        underTest.put(3, 300); // capacity 2, should evict key 2 instead

        assertThat(underTest.get(1)).isEqualTo(100);
        assertThat(underTest.get(2)).isEqualTo(-1);
        assertThat(underTest.get(3)).isEqualTo(300);
    }

    @Test
    public void shouldTreatPutOnExistingKeyAsUsage(){
        underTest.put(1, 100);
        underTest.put(2, 200);
        underTest.put(1, 111); // updates key 1, marks it most-recently-used
        underTest.put(3, 300); // capacity 2, should evict key 2

        assertThat(underTest.get(1)).isEqualTo(111);
        assertThat(underTest.get(2)).isEqualTo(-1);
        assertThat(underTest.get(3)).isEqualTo(300);
    }
}
```

Implementation (GREEN)

```
private static class Node {
    int key;
    int value;
    Node prev;
    Node next;

    Node(int key, int value) {
        this.key = key;
        this.value = value;
    }
}

private final int capacity;
private final Map<Integer, Node> map;
private final Node head;
private final Node tail;

public LRUCache(int capacity) {
    this.capacity = capacity;
    this.map = new HashMap<>();

    // sentinel nodes: head.next is always the most-recently-used real
    // node, tail.prev is always the least-recently-used. Keeping these
    // dummies means insert/remove never has to null-check a boundary.
    this.head = new Node(0, 0);
    this.tail = new Node(0, 0);
    head.next = tail;
    tail.prev = head;
}

/**
 * O(1): one map lookup, then unlink + reinsert at front (the access
```

```

    * itself counts as "use").
    */
public int get(int key) {
    Node node = map.get(key);
    if (node == null) return -1;

    remove(node);
    insertAtFront(node);
    return node.value;
}

/**
 * O(1): existing key is an update (not an eviction candidate); a new
 * key evicts the LRU node only if the cache is already full.
 */
public void put(int key, int value) {
    Node existing = map.get(key);

    if (existing != null) {
        existing.value = value;
        remove(existing);
        insertAtFront(existing);
        return;
    }

    if (map.size() == capacity) {
        Node leastRecentlyUsed = tail.prev;
        remove(leastRecentlyUsed);
        map.remove(leastRecentlyUsed.key);
    }

    Node node = new Node(key, value);
    insertAtFront(node);
    map.put(key, node);
}

```

```

}

private void remove(Node node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private void insertAtFront(Node node) {
    node.prev = head;
    node.next = head.next;
    head.next.prev = node;
    head.next = node;
}

```

Why a HashMap and a linked list, not either alone

A `HashMap` alone has no sense of order. Finding the least-recently-used entry means scanning all of them, $O(n)$. A linked list alone reorders in $O(1)$ but finding a node by key means walking it, also $O(n)$.

Combine them: the map's values ARE the list's nodes. A key finds its node in $O(1)$ through the map, and moving or unlinking that node is $O(1)$ pointer surgery, no traversal either way.

```

head ↔ node(3) ↔ node(2) ↔ node(1) ↔ tail
      most recently used           least recently used

```

Entry 12: Deep Spring

Spring Data JDBC & the Repository Pattern

Spring Data JDBC

Spring Data JDBC is a simple layer on top of JDBC. It maps database rows to Java objects **without the extra complexity of JPA/Hibernate**, such as sessions, caching, proxies, and lazy loading.

- **Immutable entities:** Entities are usually immutable, using a ``record`` or ``final`` fields. Instead of changing an object, you create a new instance when saving changes.
- **Optimistic locking:** An entity has a ``@Version`` field. When saving, Spring checks that the version hasn't changed since it was read. If someone else updated it first, the save fails rather than overwriting their changes.
- ``@DataJdbcTest``: A focused test that loads only the database-related parts of the application, such as repositories and the test database. This makes it faster and more focused than ``@SpringBootTest``.

Repository Pattern

An abstraction that lets the rest of the application access data without knowing where it comes from: a database, a cache, or somewhere else. Business logic depends on a ``Repository`` interface; the concrete implementation (JDBC, JPA, in-memory) can be swapped without affecting anything.

Reactive Spring

Traditional Spring MVC is blocking: one thread per request, and that thread sits idle while it waits on I/O operations.

Reactive programming instead treats data as a stream that gets processed and propagated as it arrives. You subscribe to that stream and react to each item rather than blocking until everything is ready.

Project Reactor is the library behind Spring's reactive stack, for building asynchronous, non-blocking applications on the producer/consumer paradigm.

- **Backpressure:** the consumer controls how much data it receives from the producer at once, instead of being flooded faster than it can process.
- The reactive stack runs on **Netty** instead of Tomcat: Netty's event-loop model handles many connections on a small pool of threads, a better fit for non-blocking I/O than one thread per request.

WebFlux is Spring's web framework built on the Reactive Streams API, the same annotations as MVC (`@RestController`, `@GetMapping`) but returning `Mono<T>` (0 or 1 item) or `Flux<T>` (0 to N items) instead of a plain object, so the whole request handling pipeline stays non-blocking end to end.

API Gateway

Usually the single entry point into a system built from multiple services. Since every request passes through it, it's a natural place to implement cross-cutting concerns once instead of repeating them in every service: security

(authentication), request monitoring, and resilience (rate limiting, circuit breaking).



Spring Cloud Gateway builds this around three concepts:

- **Route:** matches a request to a destination and forwards it.
- **Predicate:** the condition a request must match to take that route, for example a path prefix or a header value.
- **Filter:** manipulates the request or response as it passes through, adding headers, rewriting paths, handling auth flows.

Spring Data Redis

Redis is an in-memory, non-relational (key-value) data store. Because everything lives in memory instead of on disk, reads and writes are extremely fast. `spring-data-redis` gives Spring applications a `RedisTemplate` (or repository-style access) for using Redis as a cache or a lightweight message broker.

Event-Driven Architecture (Spring Cloud Stream)

Services interact indirectly, by producing and consuming events, instead of calling each other directly. A producer doesn't need to know who (or how many) consumers exist.

```
producer → [ binding ] → destination → [ binding ] → consumer  
                        (topic/queue)
```

- **Message:** the event itself, the unit of data passed around.
- **Destination:** where messages land, backed by an actual broker topic or queue (Kafka, RabbitMQ).
- **Binding:** the bridge between the application's producer/consumer code and a destination, configured declaratively rather than wired by hand.

Two common brokers, with different delivery models:

- **Kafka** (pub-sub, event streaming): consumers subscribe to a topic, and messages are retained rather than removed on read. Multiple independent consumers can read the full stream at their own pace.
- **RabbitMQ** (AMQP): producers publish events onto a queue, and each message is consumed once, then removed. A better fit for classic task distribution than for multiple independent readers of the same stream.

Spring Cloud Stream provides a binder per broker (for example `spring-cloud-stream-binder-rabbit`) so the application code talks to the same

producer/consumer abstractions regardless of which broker sits underneath; the broker-specific wiring lives in ``application.properties``.

Spring Security

A powerful, highly customizable framework that provides both authentication and access control for a Spring application.

- **Authentication:** the application establishes who the user is.
- **Authorization:** once identity is known, the application decides whether that user is allowed to do what they're asking to do.
- **Authorization server:** a dedicated service responsible for authenticating users and issuing access tokens, so applications don't each have to implement their own login flow.

Exercise: Product Except Self

Build a result array without using division.

Problem

Given an array of integers, return a new array where each index holds the product of every other element, excluding itself. Division isn't allowed, since dividing by a zero element throws an exception.

input: [1, 2, 3, 4]

output: [24, 12, 8, 6]

output[0] = $2 \times 3 \times 4 = 24$

output[1] = $1 \times 3 \times 4 = 12$

output[2] = $1 \times 2 \times 4 = 8$

output[3] = $1 \times 2 \times 3 = 6$

Tests (RED)

```
private ProductExceptSelf underTest = new ProductExceptSelf();

@Test
public void shouldReturnProductOfEveryOtherElement(){
    assertThat(underTest.productExceptSelf(new int[]{1, 2, 3, 4}))
        .containsExactly(24, 12, 8, 6);
}

@Test
public void shouldHandleThreeElements(){
    assertThat(underTest.productExceptSelf(new int[]{2, 3, 4}))
        .containsExactly(12, 8, 6);
}

@Test
public void shouldHandleASingleZero(){
    assertThat(underTest.productExceptSelf(new int[]{0, 2, 3}))
        .containsExactly(6, 0, 0);
}

@Test
public void shouldHandleMultipleZeros(){
    assertThat(underTest.productExceptSelf(new int[]{0, 0, 3}))
        .containsExactly(0, 0, 0);
}
```

Implementation (GREEN)

```
public int[] productExceptSelf(int[] nums)
{
    int n = nums.length;

    int[] lefts = new int[n];
    int runningLeft = 1;

    for (int i = 0; i < n; i++)
    {
        lefts[i] = runningLeft;
        runningLeft *= nums[i];
    }

    int[] rights = new int[n];
    int runningRight = 1;

    for (int i = n - 1; i ≥ 0; i--)
    {
        rights[i] = runningRight;
        runningRight *= nums[i];
    }

    int[] result = new int[n];
    for (int i = 0; i < n; i++)
    {
        result[i] = lefts[i] * rights[i];
    }

    return result;
}
```

Two build-up passes replace the one division-based pass:

1. `lefts[i]` is the product of everything before index `i`. Walk left to right, carrying a running product forward, and write it down *before* folding in `nums[i]`.
2. `rights[i]` is the product of everything after index `i`, the same idea walking right to left.
3. `result[i] = lefts[i] * rights[i]` combines both halves.

```
nums:    1    2    3    4
lefts:   1    1    2    6
rights: 24   12    4    1
result: 24   12    8    6
```

Why no division

The obvious shortcut is to multiply all the numbers once, then divide out `nums[i]` for each index. That breaks as soon as one element is zero, since dividing by zero isn't allowed. The zero test cases above are there to catch that shortcut.

Complexity

Time: $O(n)$, three passes over the array. Space: $O(n)$ for `lefts` and `rights`, plus the $O(n)$ output array. The two extra arrays can be dropped by writing straight into the result array and keeping the running right-to-left product in a single variable, which brings extra space down to $O(1)$.

Entry 13: Real Software Products

Automation

Building, testing, and deploying software involves a lot of repetitive steps. Doing them by hand costs time and is error-prone.

Automating pipelines is what makes it possible to ship smaller changes, more often, with more confidence.

```
commit → build → test → package → deploy
      \-----/
      automated, not manual
```

Continuous Integration (CI)

Every commit triggers an automatic build and test run.

- We get feedback in minutes.
- A broken build is visible immediately, and cheap to fix.

Continuous Delivery vs. Continuous Deployment

Both start from the same place: every build that passes CI is treated as a release candidate, ready to go to production at any time. The difference is what happens

next.

- **Continuous Delivery:** a human still decides *when* to release.
- **Continuous Deployment:** that decision is automated too. Every build that passes the pipeline goes to production with no manual gate at all.

Either way, releasing has to be a routine, low-drama event.

Automated Tests

Tests are what make CI/CD trustworthy in the first place.

- **Confidence to refactor:** a passing suite means you can restructure code and find out immediately if you broke something.
- **Executable documentation:** a test shows how the code is meant to be used, and stays accurate because it fails the moment the code drifts from it.
- **A safety net for the whole team, not just whoever wrote the code:** anyone can change a file they didn't write and trust the suite to catch what they missed.

The **testing pyramid** shapes how many of each kind you write:

```

      / \
     /   \
    / e2e \    few (they are slow)
   /----- \
  / integr. \  some (components working together)
 /----- \
/  unit      \ many (fast, isolated and cheap)
/----- \

```

- **Unit tests:** a single component in isolation, dependencies mocked or stubbed out. Fast enough to run on every save.
- **Integration tests:** real parts of the system working together (a service and a real database, for example).
- **End-to-end (E2E) tests:** drive the whole system the way a user would.

Cloud Computing

“We get computing resources dynamically, on demand.”

We rent compute, storage, and other infrastructure from a provider, and can scale that usage up or down as demand changes.

Cloud services sit at different levels of abstraction, from how much you manage yourself to how much the provider takes off your hands:

- **IaaS (Infrastructure as a Service):** raw virtual machines, storage, and networking. You control the OS and everything above it.

- **PaaS** (Platform as a Service): a managed runtime to deploy code onto; no servers to patch, but less control over the environment.
- **FaaS** (Function as a Service / serverless): you provide a single function; the platform runs it on demand and scales it invisibly.
- **SaaS** (Software as a Service): a complete application, used as-is over the network. Nothing to deploy or run at all.

Properties of cloud-native systems

- **Scalability**: handling increasing load by adding resources.
- **Loose coupling**: different parts of a system know as little as possible about each other, so one can change without breaking the rest.
- **High cohesion**: things that change together are grouped together.
- **Fault tolerance**: the system keeps working, or recovers on its own, when a part of it fails.
- **Observability**: the system's internals can be inferred from the data it exposes (logs, metrics, traces) instead of guessing.
- **Manageability**: a system can be updated, configured, or patched while it stays up and running.

Automation, testing, and these properties reinforce each other: a system that's loosely coupled and observable is also the one that's easiest to test and safe to deploy automatically.

Exercise: Best Profit

The classic "buy low, sell high" problem, solved in a single pass.

Problem

Given a non-empty array of daily stock prices, find the maximum profit from buying on one day and selling on a later day. You can only hold one share at a time, and you must buy before you sell.

```
prices: [7, 1, 5, 3, 6, 4]
```

```
buy at 1 (day 2), sell at 6 (day 5) → profit = 5
```

Implementation

```
public int maxProfit(int[] values) {  
    int minPrice = values[0];  
    int maxProfit = 0;  
  
    for (int i = 1; i < values.length; i++) {  
        maxProfit = Math.max(maxProfit, values[i] - minPrice);  
        minPrice = Math.min(minPrice, values[i]);  
    }  
  
    return maxProfit;  
}
```

One pass, two things tracked as it goes:

- **`minPrice`**: the lowest price seen *so far*, the best possible buy day up to this point.
- **`maxProfit`**: the best **`price - minPrice`** seen so far, i.e. selling today against the cheapest earlier day.

Both update every iteration. Profit is checked *before* the minimum is updated, so **`minPrice`** always reflects a day strictly earlier than today, matching the rule that you must buy before you sell.

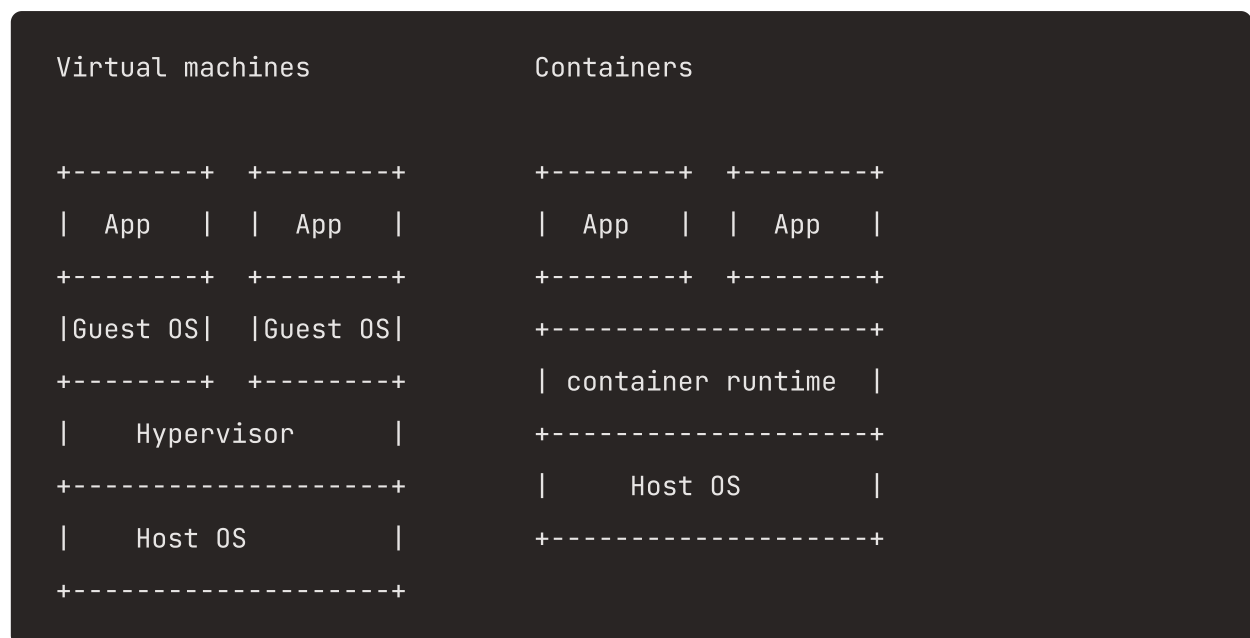
Complexity

Time: $O(n)$, one pass over the array. Space: $O(1)$, just the two running values.

Entry 14: Up & Running in Production

Virtualization vs. containers

- **Virtualization** uses a hypervisor to abstract the hardware. It can run several full operating systems on one physical machine, each isolated from the others.
- A **container** is a running instance of an image: an application packaged with everything it needs to run. Containers are much lighter than VMs because they share the host's kernel instead of booting their own OS.



This is what makes containers portable: the same image runs the same way on a laptop, a CI runner, or a cloud VM, since it carries its own dependencies and only

relies on the host kernel underneath.

Docker concepts

- **Image:** a read only, layered archive with the libraries and binaries needed to run an application. A running container adds its own writable layer on top.
- **dockerd:** the daemon running in the background that builds images, and starts, stops and manages containers.
- **Docker Engine (client/server):** the ``docker`` command line tool is a thin client. It talks to ``dockerd`` over a REST API (usually a Unix socket) to do the actual work.

```
docker CLI --REST API→ dockerd --manages→ images, containers,  
networks, volumes
```

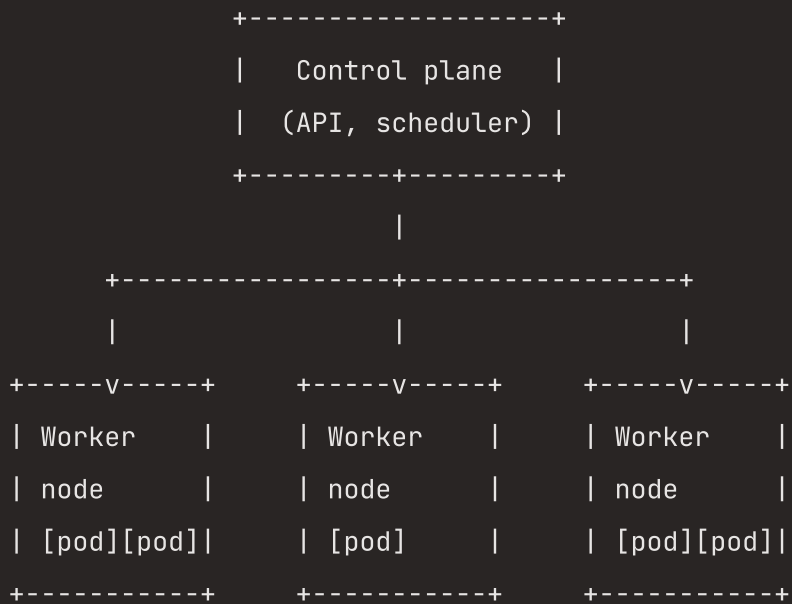
Orchestration

A single Docker host handles a handful of containers fine on its own. What it can't do is spread them across many machines, restart them when a machine dies, or scale them up under load. That's the job of an orchestrator: something that brings containers up or down as needed, from a declarative description of the desired state.

- **Cluster:** a set of nodes (machines) running containerized applications.

Kubernetes

- **Pod:** the smallest deployable unit. Wraps one or more tightly coupled containers that share networking and storage.
- **Worker node:** a VM (or physical machine) that runs pods, using three pieces: **kubelet** keeps its pods running and healthy, **kube-proxy** routes network traffic to them, and a container runtime actually runs them.
- **Control plane:** the brain of the cluster, usually running on its own dedicated nodes. Four pieces do the work: **kube-apiserver** (the front door every request goes through), **etcd** (stores the cluster's state), **kube-scheduler** (picks which node a new pod runs on), and **kube-controller-manager** (notices drift from the declared state and fixes it, like restarting a dead pod).



A few `kubectl` commands cover most of a working day:

```
kubectl get pods           # list running pods
kubectl describe pod my-pod # inspect a pod's events and status
kubectl logs my-pod        # read a container's logs
kubectl apply -f deployment.yaml # create or update resources from a
file
```

How do we create the infrastructure?

Terraform (HashiCorp) lets developers define, provision and manage infrastructure across multiple cloud providers using a declarative configuration language. It keeps a state file recording what it last created, so it knows exactly what to change on the next apply.

```
terraform init  # download providers, set up the working directory
terraform fmt   # format the configuration files
terraform plan  # preview what would change
terraform apply # create or update the real infrastructure
terraform destroy # tear it all down
```

Monitoring, tracing and distributed logging

Metrics, logs and traces are often called the three pillars of observability: together they let you tell what a system is doing without guessing. Monitoring is the practice of watching them and reacting when something looks wrong.

- **Metrics:** numeric measurements of system performance, like response time or database query counts.

- **Monitoring:** continuously gathering that data to detect problems, often with alerts when a metric crosses a threshold.
- **Tracing:** following a single request as it moves across multiple services, to find bottlenecks or failures in a distributed system. Each request carries a trace ID that ties its spans together across services.
- **Distributed logging:** collecting logs from every service into one searchable place, so a single request can be followed through them by that same trace ID.

Prometheus

An open source monitoring tool that pulls metrics from configured targets at regular intervals and stores them as time series, queryable with PromQL.

Grafana

An open source dashboard and visualization tool. It can use Prometheus (among other systems) as a data source to turn those metrics into readable charts and alerts.

Exercise: Group Anagrams

Two ways to group words that are anagrams of each other, both hashing on a shared key.

Problem

Given a list of strings, group the ones that are anagrams of each other (same letters, any order) into buckets.

```
input: ["eat", "tea", "tan", "ate", "nat", "bat"]  
output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

Tests (RED)

```
private GroupAnagrams underTest = new GroupAnagrams();

@Test
public void shouldGroupAnagrams() {
    assertThat(underTest.groupAnagrams(List.of("eat", "tea", "tan",
        "ate", "nat", "bat"))).values()
        .containsExactlyInAnyOrder(
            List.of("eat", "tea", "ate"),
            List.of("tan", "nat"),
            List.of("bat")
        );
}

@Test
public void shouldGroupAnagramsUsingSort() {
    assertThat(underTest.groupAnagramsSortedKeys(List.of("eat", "tea",
        "tan", "ate", "nat", "bat"))).values()
        .containsExactlyInAnyOrder(
            List.of("eat", "tea", "ate"),
            List.of("tan", "nat"),
            List.of("bat")
        );
}
```

Implementation (GREEN)

Both implementations group words by computing a key that is identical for every anagram of the same word, and different otherwise.

```

public Map<String, List<String>> groupAnagramsSortedKeys(List<String>
anagrams) {
    Map<String, List<String>> groupedAnagrams = new HashMap<>();

    for (String anagram : anagrams) {
        char[] chars = anagram.toCharArray();
        Arrays.sort(chars);
        String freqKey = new String(chars);

        if (groupedAnagrams.containsKey(freqKey)) {
            groupedAnagrams.get(freqKey).add(anagram);
        } else {
            groupedAnagrams.put(freqKey, new ArrayList<>
(List.of(anagram)));
        }
    }

    return groupedAnagrams;
}

```

Sorting a word's characters gives a key that is identical for every anagram of it:
`"eat"`, `"tea"` and `"ate"` all sort to `"aet"`.

```

public Map<String, List<String>> groupAnagrams(List<String> anagrams) {
    Map<String, List<String>> groupedAnagrams = new HashMap<>();

    for (String anagram : anagrams) {
        String freqKey = Arrays.toString(charFrequencies(anagram));

        if (groupedAnagrams.containsKey(freqKey)) {
            groupedAnagrams.get(freqKey).add(anagram);
        } else {
            groupedAnagrams.put(freqKey, new ArrayList<>
(List.of(anagram)));
        }
    }

    return groupedAnagrams;
}

private int[] charFrequencies(String word) {
    int[] chars = new int[128];

    for (int i = 0; i < word.length(); i++) {
        chars[word.charAt(i)]++;
    }

    return chars;
}

```

The second version skips sorting: a 128 slot array counting how many times each character appears is just as good a key, since anagrams always produce the same counts.

Complexity

Let n be the number of words and k the length of the longest one.

- **`groupAnagramsSortedKeys`**: sorting each word costs $O(k \log k)$, so $O(n * k \log k)$ time overall.
- **`groupAnagrams`**: counting characters is $O(k)$ per word, so $O(n * k)$ time overall, faster since it avoids the sort.

Both are $O(n * k)$ space: every word ends up stored once, in some group.

Entry 15: Relational Databases

Modeling entities and relationships

Our domain: teachers, students, and courses.

- A teacher teaches many courses, but each course has one teacher: **one-to-many**.
- A student can enroll in many courses, and a course can have many students: **many-to-many**.

```
Teacher (1) ---- teaches ----> (N) Course
```

```
Student (N) <--- enrolled in ---> (N) Course
```

A many-to-many relationship can't be stored directly in two tables. We need a junction (or join) table, holding one row per pairing.

```
Student (1) ---- (N) Enrollment (N) ---- (1) Course
```

Why normalize

Put everything into one flat table, and duplication follows.

student	course	teacher	teacher_department
Ana	Math 101	Diaz	Science
Ana	Physics	Osei	Science
Ben	Math 101	Diaz	Science

Diaz and Science show up in every row where Diaz teaches.

That duplication causes three problems: update, insertion, and deletion anomalies.

Normalization is the process of splitting a table like this into smaller ones so each fact is stored exactly once. The formal steps are called normal forms.

1NF: atomic values

Every column must hold a single value, never a list.

2NF: no partial dependency

Applies once a table has a composite key (more than one column together forming the key). Every other column must depend on the whole key, not just part of it.

3NF: no transitive dependency

A non-key column can't depend on another non-key column instead of the key.

Applied to the flat table above: `teacher` depends only on `course`, not on the full `(student, course)` key, so 2NF moves it out. `teacher_department`

depends on `teacher`, a non-key column, not directly on `course`, so 3NF moves it out too.

After all three steps:

```
Teacher(id, name, department)
Course(id, title, teacher_id)
Student(id, name)
Enrollment(student_id, course_id)
```

Each fact now lives in exactly one place. The cost: reading "Ana's courses with teacher department" now takes three joins instead of one flat read.

That's the normalized versus denormalized trade-off. Normalized favors correctness and cheap, safe writes.

Denormalized favors fast reads but might lose consistency.

Querying across the tables

SQL is built on relational algebra. Some of its clauses map directly onto operators from it:

- σ (selection): filter rows, `WHERE`
- π (projection): pick columns, `SELECT`
- $\bowtie$ (join): combine relations on a condition, most often a foreign key matching the primary key it references, `JOIN`

- $\cap$ (intersection): rows present in both relations, both relations must have the same columns, **INTERSECT**

Projection, just the student names:

π name (Student)

```
SELECT name FROM Student;
```

Join, each student with their teacher and course:

π student.name, teacher.name, course.title (Student $\bowtie$ (id=student_id, Enrollment $\bowtie$ (id=course_id, Course $\bowtie$ (id=teacher_id, Teacher))

```
SELECT s.name AS student, t.name AS teacher, c.title AS course
FROM Student s
JOIN Enrollment e ON s.id = e.student_id
JOIN Course c ON e.course_id = c.id
JOIN Teacher t ON c.teacher_id = t.id;
```

Intersection, students enrolled in both Math and Physics:

π student_id (σ course_id='Math101' Enrollment) $\cap$ π student_id (σ course_id='Physics101' Enrollment)

Math students		Physics students
+-----+		+-----+
Ben		
	+---+-----+---+	
	Ana	
+-----+		+-----+

^

enrolled in both

```
SELECT s.name
FROM Student s
WHERE s.id IN (SELECT e0.student_id
               FROM Enrollment e0
               WHERE e0.course_id = 'Math101'
               INTERSECT
               SELECT e1.student_id
               FROM Enrollment e1
               WHERE e1.course_id = 'Physics101')
```

Exercise: Min and Max

Picked a light exercise since this entry's theory already covers a lot of ground.

Problem

Write `minAndMax(int[] numbers)`: return an array of two elements, the smallest and the largest number in the input, in that order.

```
{43,54,56,22,3,67,1,99,21} → {1, 99}
```

Tests (RED)

```
@Test
public void shouldReturnMinAndMax()
{
    Day2 day2 = new Day2();
    assertThat(day2.minAndMax(new int[]
{43,54,56,22,3,67,1,99,21})).isEqualTo(new int[]{1,99});
}
```

Implementation (GREEN)

```
public int[] minAndMax(int[] numbers)
{
    int[] minMax = new int[]{numbers[0], numbers[0]};

    for (int i = 1; i < numbers.length; i++)
    {
        if (numbers[i] < minMax[0]) minMax[0] = numbers[i];
        if (numbers[i] > minMax[1]) minMax[1] = numbers[i];
    }

    return minMax;
}
```

Why one pass is enough

Time complexity: $O(N)$, every number is visited once

Space complexity: $O(1)$, just two variables besides the input

Entry 16: Networking Basics

Protocol

A protocol is an agreed format for how data is structured and sent over a network. Two systems know how to understand each other's bytes.

The OSI model

It splits network communication into seven layers, each one has a single responsibility.

7	Application	HTTP, FTP, SMTP
6	Presentation	data formatting, encryption
5	Session	set up, coordinate, end communication
4	Transport	TCP, UDP
3	Network	IP addressing, routing
2	Data link	LLC, MAC
1	Physical	raw bits over the wire

Physical layer

Transmits raw bits over a physical medium: cables, radio, fiber.

Data link layer

Provides a reliable link between two devices. Splits into two sublayers:

- **LLC** (Logical Link Control): flow control and error checking.
- **MAC** (Media Access Control): identifies devices on the same network.

Network layer

Gives every device a unique address and routes data between networks.

- IPv4 addresses are 32 bits.
- IPv6 addresses are 128 bits.

Transport layer

Handles two way communication between applications, end to end.

- **TCP**: reliable, ordered, connection based.
- **UDP**: fast, no delivery guarantee.

Session layer

Sets up, coordinates, and ends communication sessions between two devices.

Presentation layer

Formats data into something the application layer understands: encryption, compression, encoding.

Application layer

The interface between the application and the network. Protocols like HTTP, FTP, and SMTP live here.

Firewalls

A firewall controls incoming and outgoing traffic, based on a set of rules.

AWS security groups

On AWS, we set up **security groups**: a firewall attached to an instance, allowing traffic in and out by rule.

```
Internet
  |
  v
security group: web-sg
  allow 443 (https) from 0.0.0.0/0
  allow 22 (ssh)   from 203.0.113.0/24
  deny  everything else
  |
  v
EC2 instance
```

```
resource "aws_security_group" "web_sg" {
  name          = "web-sg"
  description   = "Allow HTTPS and SSH"
  vpc_id        = aws_vpc.main.id

  ingress {
    description = "HTTPS from anywhere"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "SSH from office"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.0/24"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Everything not explicitly allowed by an ingress rule is denied by default.

NAT

NAT (Network Address Translation) maps addresses from one network to another.

- **Source NAT:** lets hosts inside a private network reach the internet, by rewriting their private address to a public one.
- **Destination NAT:** forwards incoming traffic to a specific host inside a private network.

Subnets

A subnet splits one network into smaller networks.

An IP address is split into two parts using a mask: the network part and the host part.

A `/24` mask means the first 24 bits identify the subnet, leaving 8 bits for hosts.

121.128.0.1	a host on the subnet
121.128.0.0	network address
121.128.0.255	broadcast address

The network address marks the subnet itself, and the broadcast address reaches every host on it. Neither one can be assigned to a device, so a `/24` subnet has 254 usable host addresses, not 256.

Exercise: Unique Characters

A quick one, reusing the frequency table from previous exercises.

Problem

Write `everyCharIsUnique(String word)`: return `true` if no character repeats in the string.

```
everyCharIsUnique("abc")    → true  
everyCharIsUnique("abcas") → false   ('a' repeats)
```

Tests (RED)

```
@Test  
public void shouldCheckEveryCharIsUnique(){  
    Day2 day2 = new Day2();  
  
    assertThat(day2.everyCharIsUnique("abc")).isTrue();  
    assertThat(day2.everyCharIsUnique("abcas")).isFalse();  
}
```

Implementation (GREEN)

```
public boolean everyCharIsUnique(String word)
{
    int[] frequencies = freqTable(word);

    for (int i = 0; i < frequencies.length; i++)
        if (frequencies[i] > 1) return false;

    return true;
}
```

`**freqTable**` builds a 128 slot array counting how many times each character appears. If every slot holds at most 1, every character in the word is unique.

```
public int[] freqTable(String word)
{
    char[] chars = word.toCharArray();
    int[] result = new int[128];

    for (int i = 0; i < chars.length; i++)
    {
        result[chars[i]]++;
    }

    return result;
}
```

Complexity

Time complexity: $O(N)$

Space complexity: $O(1)$

Entry 17: Operating Systems and Linux

What an operating system does

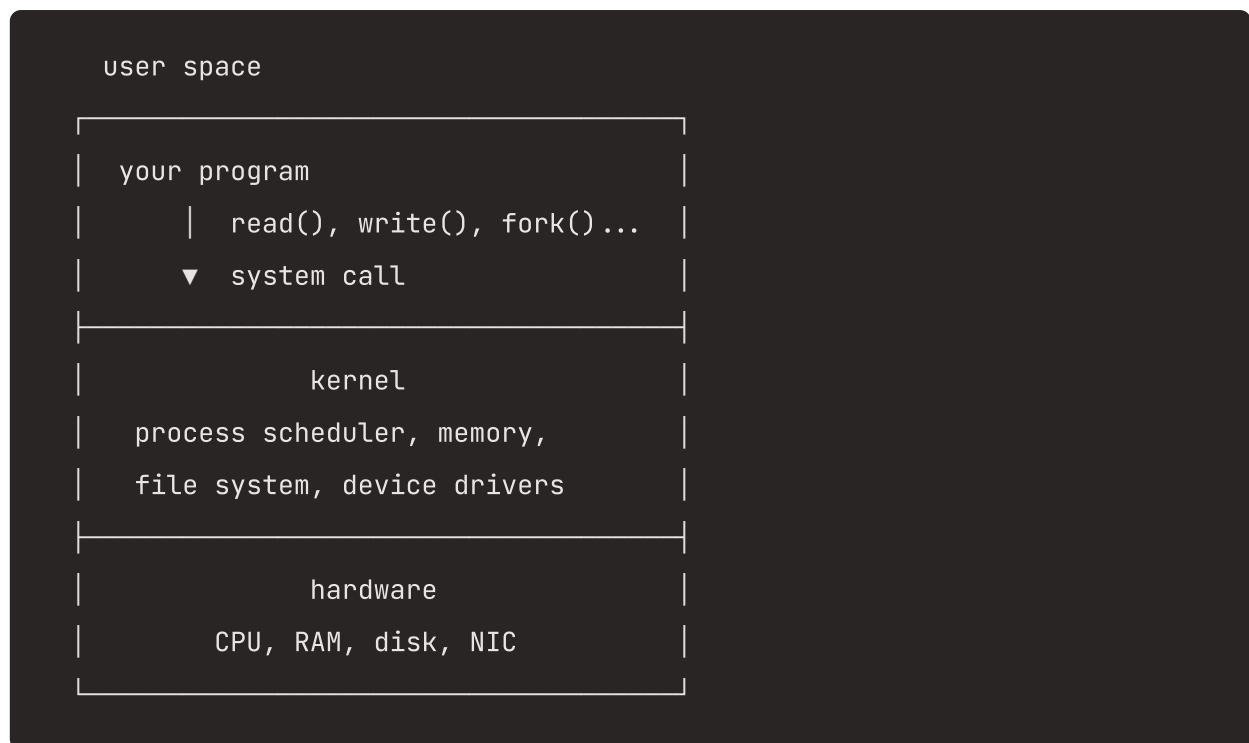
An **operating system (OS)** manages the CPU, memory, storage, and I/O devices, and gives each program an interface to use them.

Without it, every application would need to know how to manage a disk controller, network cards, and CPU scheduler directly.

Kernel space and user space

The **kernel** is the core of the OS. It talks to hardware and decides which process gets the CPU next.

Everything else (your browser, your terminal, your Java program) runs in **user space** with restricted privileges. They use **system calls** to ask the kernel what they need.



Process and threads, quickly

A **process** is a running program: its own memory space, a set of open files and execution state. The OS creates processes, schedules them on the CPU, and cleans up after they exit.

A **thread** is a unit of execution inside a process. Threads in the same process share memory.

Some basics

- **Shell:** the program that reads the commands you type and asks the kernel to run them. Bash and zsh are shells.

- **File descriptor:** a small integer a process uses to refer to an open file, socket, or pipe.
- **Standard streams:** every process starts with three file descriptors already open: ``stdin`` (0, input), ``stdout`` (1, normal output), ``stderr`` (2, errors).

Useful and popular commands: grep and pipes

``grep`` is older than most tools still in daily use. Its name comes from an ``ed`` (a very old line editor) command: ``g/re/p``, "globally search for a regular expression and print matching lines". The command outlived the editor it was named after.

The **pipe** (``|``) connects the standard output of one process directly to the standard input of the next, with nothing written to disk in between.

```
ps aux | grep java
```

```
ps aux  —stdout—►  grep java  —stdout—►  terminal
(every process)      (only matches)
```

Nothing here is specific to ``grep``, the shell can chain any two programs this way as long as one writes to stdout and the other reads stdin.

Inter-process communication (IPC)

Processes don't share memory by default. The OS provides ways for them to exchange data:

- **Pipes:** a one way byte stream between two related processes (like the shell pipe above).
- **Shared memory:** the fastest option, the kernel maps the same block of memory into more than one process, no copying needed. It needs synchronization, the OS doesn't order the accesses for you.
- **Sockets:** like a pipe, but works between unrelated processes, across machines, over a network.
- **Signals:** a small integer sent to a process to notify it about something (`SIGKILL`, `SIGTERM`).

A pipe example implemented in C

`pipe()` asks the kernel for two file descriptors: one for reading, one for writing.

It becomes real IPC once combined with `fork()`, which clones the calling process.

The child inherits copies of the parent's open file descriptors, including both ends of the pipe.

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pipe(fd); // fd[0] = read end, fd[1] = write end

    pid_t pid = fork(); // clones this process

    if (pid == 0) {
        // I am in the child process
        close(fd[1]); // not writing, close that end
        char buffer[64] = {0};
        read(fd[0], buffer, sizeof(buffer));
        printf("child received: %s\n", buffer);
        close(fd[0]);
    } else {
        // I am in the parent process
        close(fd[0]); // not reading, close that end
        char *msg = "hello from parent";
        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);
    }

    return 0;
}

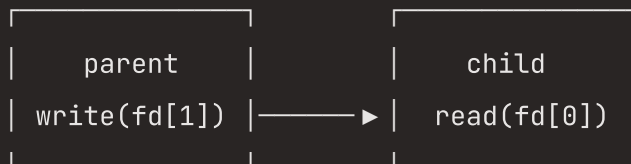
```

Save this as a `.c` file and compile it with `gcc`, no special flags needed since `pipe()`, `fork()`, `read()`, and `write()` are all standard POSIX functions declared in `unistd.h`:

```
gcc pipe_demo.c -o pipe_demo
./pipe_demo
# child received: hello from parent
```

```
pipe(fd)
fd[1] write —▶ fd[0] read
```

```
fork()
```



Exercise: Palindrome Permutation

Reuses ``freqTable``, same helper as the unique characters exercise, on a different question.

Problem

Write ``isAPalindromePermutation(String input)``: return ``true`` if the characters of the string can be rearranged into some palindrome. Case and spaces don't count.

```
isAPalindromePermutation("Tact Coa")      → true   ("taco cat" is a
palindrome)
isAPalindromePermutation("This is a test") → false
```

Why frequency counts are enough

A string's letters can be rearranged into a palindrome when at most one character has an odd count.

Tests (RED)

```
@Test
public void shouldCheckPalindromePermutations(){
    Day3 d3 = new Day3();

    assertThat(d3.isAPalindromePermutation("Tact Coa")).isTrue();
    assertThat(d3.isAPalindromePermutation("tact coa")).isTrue();
    assertThat(d3.isAPalindromePermutation("Taco Cat")).isTrue();
    assertThat(d3.isAPalindromePermutation("This is a test")).isFalse();
    assertThat(d3.isAPalindromePermutation("abc")).isFalse();
    assertThat(d3.isAPalindromePermutation("aab")).isTrue();
    assertThat(d3.isAPalindromePermutation("aabb")).isTrue();
    assertThat(d3.isAPalindromePermutation("aabbc")).isTrue();
    assertThat(d3.isAPalindromePermutation("aabbcd")).isFalse();
    assertThat(d3.isAPalindromePermutation("racecar")).isTrue();
    assertThat(d3.isAPalindromePermutation("")).isTrue();
    assertThat(d3.isAPalindromePermutation("a")).isTrue();
}
```

Implementation (GREEN)

```
public boolean isAPalindromePermutation(String input)
{
    Day2 day2 = new Day2();

    int[] frequencies = day2.freqTable(input.replace(" ",
    "").toLowerCase());

    int oddsFrequencies = 0;

    for (int i = 0; i < frequencies.length; i++)
    {
        if ((frequencies[i] % 2) != 0) oddsFrequencies++;
        if (oddsFrequencies > 1) return false;
    }

    return true;
}
```

Complexity

```
n = length of input
k = size of the frequency table (128, constant)

freqTable build:    O(n)
frequency scan:    O(k)

O(n + k)  k is constant  ⇒  O(n)
Space: O(k)  ⇒  O(1)
```

Entry 18: TDCG, Using AI safely

The problem with a bare prompt

AI can generate the implementation I need much faster than I can write it by hand. Faster is not the same as correct, and it is not the same as safe. An implementation that looks right but was never asked to prove itself is a promise, not a guarantee.

Test-Driven Code Generation (TDCG) is my method for keeping AI-assisted work honest. It takes the core discipline of TDD and applies it to a workflow where the AI is the one writing both the test and the implementation.

“The AI generates. The human decides what is right and what is not.”

The loop

1. SPECIFY - agree on one behavior, in plain language
2. RED - AI generates failing tests, human reviews and approves
3. PROMPT - the approved tests become the spec for the implementation
4. GREEN - AI generates the implementation, tests pass
5. REVIEW - check naming, responsibilities, duplication, dead code
6. REFACTOR - apply what review found, tests stay green throughout
7. COMMIT - one small, focused commit with a clear why

Why each step earns its place

- **SPECIFY** forces one small behavior at a time.
- **RED** happens before any implementation exists. If a human never reads the test, they never really own the specification; they are just trusting the AI's guess at what "done" means.
- **PROMPT** hands the approved tests back to the AI as the spec, along with a reference file or existing pattern to follow.
- **GREEN** only passes because the tests already say what correct looks like.
- **REVIEW** Passing tests say nothing about naming, duplication, or whether the design fits the rest of the codebase, so an experienced human still has to read the implementation.
- **REFACTOR** Change what we found during the review: renaming, removing duplication, simplifying. Tests stay green throughout; if a refactor breaks one, the refactor is wrong, not the test.
- **COMMIT** stays small on purpose. A small, tested, reviewed change is easy to understand and easy to undo if something goes wrong.

Where it fits, and where to be careful

Works well when:

- the codebase already has clear patterns to follow
- what to add is clear, even when the implementation itself is not trivial

Be more careful with:

- greenfield design, there is no existing pattern for the AI to anchor to, so the specify step has more room to drift
- performance and security-sensitive code, since passing tests prove the behavior asked for, not that it is fast enough or safe against a determined attacker

The habit, not just the steps

The point is to stop generated implementations from getting into a codebase without proof that it is doing what it needs to do, and without a human who actually read them. TDCG just turns that into a habit that is hard to skip by accident.

Exercise: String Reverse

A short one because it is Friday and I need a beer.

Problem

Write `stringReverse(String word)`: return the string with its characters in reverse order.

```
stringReverse("abc") → "cba"
```

Tests (RED)

```
@Test
public void shouldReverseAString(){
    Day3 d3 = new Day3();

    assertThat(d3.stringReverse("abc")).isEqualTo("cba");
}
```

Implementation (GREEN)

```
public String stringReverse(String word)
{
    StringBuffer buffer = new StringBuffer();

    for (int i = word.length() - 1; i ≥ 0; i--)
    {
        buffer.append(word.charAt(i));
    }

    return buffer.toString();
}
```

The loop walks the input backwards, one character at a time, appending each one as it goes. `StringBuffer` is the thread-safe twin of `StringBuilder`, same idea, synchronized methods.

Complexity

Time complexity: $O(N)$

Space complexity: $O(N)$

Entry 19: Software Paradigms

Why paradigms matter

A paradigm defines how we organize the solution to a problem. How you break a problem into smaller parts, how those parts work together.

Most languages let you work and combine different paradigms. But the three below have had a huge influence, and you can still see them in everyday work.

Structured Programming

Back when `GOTO` was common, Edsger Dijkstra argued that programs were difficult to understand when their control flow was too tangled. If a program could jump anywhere, understanding one line might require tracing through many other parts of the program.

Structured programming replaced these arbitrary jumps with three basic control structures:

- **Sequence:** run statements in order.
- **Selection:** choose between paths with `if`/`else`.
- **Iteration:** repeat code with `while`/`for`.

These structures make programs easier to understand because each block has a clear beginning and end.

```
// unstructured: where you end up depends on history, not just this line
start:
if (done) goto end;
// ... do work ...
goto start;
end:

// structured: sequence, selection and iteration, each provable in
isolation
while (!done) {
    // ... do work ...
}
```

Object-Oriented Programming

OOP takes decomposition a step further by grouping **data and the behaviour that works on that data** into one unit: an **object**.

The object hides its internal details behind an interface. Code using the object only needs to know **what it can do**, not **how it does it**.

- **Abstraction and encapsulation** hide an object's internal details.
- **Polymorphism** allows different implementations to be used through the same interface. Code that works with the interface does not need to change when a different implementation is plugged in.

- **Dependency Inversion** takes this idea further: code should depend on abstractions rather than specific implementations. This makes implementations easier to replace and test.

OOP is not free. Abstractions can make implementations easier to change, but they can also make it harder to understand.

Functional Programming

Functional programming has its roots in **lambda calculus**, a model of computation developed by Alonzo Church in the 1930s, before electronic computers existed.

Lisp brought these ideas into a practical programming language and treated **functions as first-class values**. This means functions can be assigned to variables, passed as arguments, and returned from other functions.

The core idea is to use **pure functions** with **immutable data**:

- A pure function always produces the same output for the same input.
- It does not change anything outside itself.
- Immutable data cannot be changed after it is created.

```
// imperative: a shared mutable accumulator, mutated across iterations
List<Integer> squares = new ArrayList<>();
for (int n : numbers) {
    if (n % 2 == 0) squares.add(n * n);
}

// functional: each step produces a new value, nothing is mutated in
// place
List<Integer> squares = numbers.stream()
    .filter(n → n % 2 == 0)
    .map(n → n * n)
    .toList();
```

Both produce the same result, but the imperative version mutates a shared list, while the functional version transforms values into a new result.

Avoiding shared mutable state can also make concurrent code easier to reason about, without relying as heavily on locks such as `synchronized` or `AtomicInteger`.

Picking a Paradigm

None of these paradigms replaced the others; they work well together. A typical Java service might use structured programming within methods, OOP to organise modules, and functional programming for transformations that don't need shared state.

What matters is knowing **which paradigm you're using and why**, rather than trying to use one everywhere.

Exercise: Frequency Table, Two Ways

Problem

Build a frequency table for the characters in a string.

What would the functional approach look like?

```
freqTableStreams("AABBBC") → table['A'] = 2, table['B'] = 3,  
table['C'] = 1
```

Tests (RED)

```
@Test  
public void shouldBuildAFreqTableWithStreams() {  
    Day2 day2 = new Day2();  
  
    int[] table = day2.freqTableStreams("AABBBC");  
  
    assertThat(table['A']).isEqualTo(2);  
    assertThat(table['B']).isEqualTo(3);  
    assertThat(table['C']).isEqualTo(1);  
}
```

The tests should check that the new implementation behaves the same as the existing one.

Implementation (GREEN)

```
public int[] freqTableStreams(String word) {  
    int[] result = new int[128];  
    word.chars().forEach(c → result[c]++);  
    return result;  
}
```

Compare it with the original:

```
public int[] freqTable(String word) {  
    char[] chars = word.toCharArray();  
    int[] result = new int[128];  
  
    for (int i = 0; i < chars.length; i++) {  
        result[chars[i]]++;  
    }  
  
    return result;  
}
```

Was it worth it?

Not really. The stream version removes the explicit loop, but it still mutates `result` on every element.

A more functional version could return a `Map`:

```
Map<Character, Long> counts = word.chars()  
    .mapToObj(c → (char) c)  
    .collect(Collectors.groupingBy(c → c, Collectors.counting()));
```

But that changes the return type and would require changes to the code using it.

The takeaway: use streams when they make a transformation clearer, not just because a loop can be rewritten as one.

Complexity

Time: $O(N)$

Space: $O(1)$, the output array always has 128 elements

Entry 20: Software Design, Writing It Well & Working Together

Why bad design costs more over time

In a poorly designed system, every change becomes harder and more expensive because developers spend more time fixing bugs, working around fragile assumptions and resolving dependencies.

Adding more developers or producing more output with AI tools doesn't solve the problem. If the design is bad, you simply create more complexity, faster.

What writing it well actually means

Writing well means it's easy to read, understand and change. A few practices make the biggest difference:

- **Keep It Simple (KISS):** use the simplest solution that works. Unnecessary cleverness creates extra cost.
- **Meaningful names:** names should make a function's purpose clear without needing comments.

```
// unclear: what is 0.05, and why are we multiplying?
double calc(double a, double b) {
    return a * b * 0.05;
}

// clear: the name carries the intent
double calculateTax(double price, double taxRate) {
    return price * taxRate;
}
```

- **Small, single-purpose functions:** each function should do one thing well. Smaller units are easier to understand, change and test.

The goal is to reduce the amount of context someone needs to understand when making a change in the future.

Reviews and pair programming

Two people spot different problems, which is why reviews catch bugs, unclear names and missing edge cases that the author may miss. They also spread knowledge across the team.

Pair programming takes this further by reviewing it as it's written. It's especially useful for junior developers, who can learn good habits and context directly from more experienced teammates.

Teams with good pair programming practices might not need a separate review at all. This practice shows good communication and collaboration between the team members. These are key to having a successful project.

Working as a team: the Agile values

Good practices and reviewing well only make sense if every member of the team is clear about the shared goal and pushing in the same direction.

In 2001, the Agile Manifesto defined four values for how teams should work:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

The things on the right still have value but in case of conflict we should prioritise what is on the left. For example, a project plan should never be more important than delivering working software.

Exercise: Arrays.merge Test Retrofit

Problem

Merge two already-sorted `int[]` arrays into a single one without using a library.

```
merge(new int[]{1, 3, 5}, new int[]{2, 4, 6}) → [1, 2, 3, 4, 5, 6]
```

A design-debt example first

The implementation was written first, and only now are the tests catching up.

Existing implementation

```
public int[] merge(int[] arr0, int[] arr1) {
    int[] merged = new int[arr0.length + arr1.length];
    int a = 0, b = 0, z = 0;

    while (a < arr0.length && b < arr1.length) {
        if (arr0[a] < arr1[b]) {
            merged[z] = arr0[a];
            a++;
        } else {
            merged[z] = arr1[b];
            b++;
        }
        z++;
    }

    while (a < arr0.length) { merged[z] = arr0[a]; a++; z++; }
    while (b < arr1.length) { merged[z] = arr1[b]; b++; z++; }

    return merged;
}
```

Tests written after the fact

Without a spec, you need to think carefully about what could go wrong. For a two-pointer merge, tests should cover cases like:

- Both arrays contribute elements.
- One array runs out first.
- The edge cases around these situations.

```
@Test
public void shouldMergeTwoArraysOfEqualLength() {
    assertThat(arrays.merge(new int[]{1, 3, 5}, new int[]{2, 4, 6}))
        .containsExactly(1, 2, 3, 4, 5, 6);
}

@Test
public void shouldMergeArraysOfDifferentLengths() {
    assertThat(arrays.merge(new int[]{1, 2}, new int[]{3, 4, 5, 6}))
        .containsExactly(1, 2, 3, 4, 5, 6);
}

@Test
public void shouldMergeWhenOneArrayIsEmpty() {
    assertThat(arrays.merge(new int[]{}, new int[]{1, 2, 3}))
        .containsExactly(1, 2, 3);
}

@Test
public void shouldKeepDuplicatesWhenMerging() {
    assertThat(arrays.merge(new int[]{1, 2, 2}, new int[]{2, 3}))
        .containsExactly(1, 2, 2, 2, 3);
}

@Test
public void shouldMergeNegativeNumbers() {
    assertThat(arrays.merge(new int[]{-5, -1, 3}, new int[]{-3, 0, 2}))
        .containsExactly(-5, -3, -1, 0, 2, 3);
}
```

Complexity

Time: $O(n + m)$, every element from both arrays is visited once

Space: $O(n + m)$, for the output array